

Gestão do Ciclo de Vida da Aplicação

Fundamentos • Processos • Requisitos
Prototipação • Projeto de software

40 páginas / 72 questões

Exemplos resolvidos + simulado + gabarito comentado

Leia, pratique sem consulta e confira as respostas apenas no final. Para justificar decisões: conceito, evidência, consequência e melhoria.

Base: 11 aulas, Guia do Scrum 2020 e quatro questionários do arquivo Slides.zip. Exercícios adaptados e autorais, sem previsão ou gabarito oficial da prova.

Elaborada em 13 de setembro de 2026.

Roteiro de estudo e mapa da apostila

O material exige mais do que memorização. Os questionários pedem comparar abordagens, interpretar situações, escrever requisitos, analisar código e justificar escolhas. No questionário 3, os feedbacks explicitam que uma resposta precisa conectar contexto, risco, custo e aprendizado.

Leitura	Assunto	Páginas
1	Fundamentos e evolução dos processos	3 a 5
2	XP e Scrum	6 a 9
3	Kanban e métricas de fluxo	10 e 11
4	Requisitos, histórias e casos de uso	12 a 14
5	MVP e testes A/B	15 e 16
6	Prototipação e testes com usuários	17 a 19
7	Propriedades, métricas e princípios de projeto	20 a 25
8	Integração e revisão de véspera	26 e 27
9	Simulado de 12 questões	28 e 29
10	Fontes e ressalvas do material	30
11	Gabarito comentado	31 em diante

Se você tem apenas uma noite

Reserve cerca de 4 a 5 horas de estudo efetivo: 60 minutos para projeto e métricas; 60 para Scrum, XP e Kanban; 45 para requisitos, MVP e A/B; 40 para prototipação; 45 para o simulado e correção. Faça pausas entre blocos e reduza a releitura dos assuntos que já consegue explicar.

Se tiver mais tempo, siga a ordem da apostila e resolva as 60 questões dos capítulos. Se tiver menos, priorize os exemplos resolvidos, as caixas de atenção e o simulado. Volte à teoria sempre que errar uma justificativa.

Como construir uma boa resposta

Use quatro movimentos: **nomear o conceito**; **mostrar onde ele aparece**; **explicar seu efeito**; **propor uma ação coerente**. Para cálculos, acrescente fórmula, substituição dos valores, unidade e interpretação.

Uma resposta como “viola responsabilidade única” identifica o assunto, mas não demonstra aplicação. Melhor: “O método mistura interface, persistência e regra de negócio. Por isso, mudanças independentes atingem o mesmo trecho e dificultam testes. Separaria a interface, um serviço de aplicação e um repositório.”

PRIORIDADE NÃO É PREVISÃO: todos os blocos aparecem no material. A ordem de revisão sugerida se baseia na densidade e no tipo das questões, não em informação sobre o conteúdo efetivo da prova.

1. Fundamentos de Engenharia de Software

O problema vai além de programar

Engenharia de Software organiza a construção, a avaliação e a evolução de sistemas. Inclui requisitos, projeto, implementação, testes, implantação e manutenção. Um programa pode executar sem falhas aparentes e ainda ser um produto ruim: basta resolver o problema errado, ser difícil de usar ou custar demais para evoluir.

Os exemplos históricos da aula 1 mostram por que técnicas e organização importam. A chamada crise do software envolve dificuldade para entregar sistemas complexos com qualidade, prazo e custo aceitáveis. A conferência da OTAN de 1968 é um marco abordado no material.

Não existe uma solução universal

A discussão de Brooks sobre “não existir bala de prata” destaca dificuldades essenciais. **Complexidade:** muitas partes, estados e interações. **Conformidade:** adaptação a regras, sistemas e organizações existentes. **Mudança:** necessidades e ambiente evoluem. **Invisibilidade:** software não possui uma representação física única que revele toda a sua estrutura.

Uma ferramenta pode reduzir esforço de implementação sem eliminar a dificuldade de decidir o que construir, lidar com conflitos ou manter coerência. Evite respostas absolutas como “a técnica X resolve qualquer projeto”.

Requisitos funcionais e não funcionais

Requisitos funcionais descrevem serviços ou comportamentos: consultar saldo, cancelar reserva ou emitir relatório. Requisitos não funcionais expressam qualidades e restrições: desempenho, disponibilidade, segurança, capacidade, interoperabilidade e manutenibilidade.

Compare “o sistema permite consultar uma reserva” com “a consulta responde em até 2 segundos para 95% das requisições sob a carga definida”. O segundo enunciado é verificável; “o sistema deve ser rápido” é vago. A classificação depende da formulação: um objetivo de segurança pode exigir comportamentos funcionais específicos, como bloquear uma conta após tentativas inválidas.

Sistemas A, B e C

Na classificação apresentada na aula: **A (Acute)** são sistemas de missão crítica, com falhas de consequências muito graves, como controle de voo ou de equipamento médico; **B (Business)** sustentam atividades importantes, como matrícula, folha de pagamento e comércio eletrônico; **C (casuais)** são pequenos, de baixo impacto e por vezes descartáveis.

A classificação depende do contexto e das consequências da falha, não apenas da linguagem ou do tamanho do código. Sistemas B precisam de engenharia consistente; sistemas C podem sofrer com complexidade desnecessária, o chamado overengineering.

Pratique antes de continuar

Q01. Uma biblioteca pede “renovar empréstimos”, “atender 2.000 usuários simultâneos” e “ser fácil de usar”. Classifique os três requisitos e reescreva o último de modo verificável.

Q02. Classifique um script pessoal descartável, um sistema de matrícula e um controlador de freios. Justifique considerando consequências de falha.

1.1 Testes, manutenção e trabalho em equipe

O que um teste permite concluir

Um teste executa o software em condições definidas e compara o resultado observado ao esperado. **Testes revelam falhas nos casos exercitados; não demonstram, em geral, que todas as entradas, estados, sequências e ambientes possíveis estão corretos.** Ausência de falhas observadas não equivale a ausência de defeitos.

Testes de **unidade** verificam partes pequenas de forma isolada; de **integração** verificam a colaboração entre componentes; de **sistema** avaliam o comportamento do sistema completo. A pirâmide de testes orienta uma base ampla de testes menores e rápidos, complementada por menos testes amplos e custosos. É uma orientação de estratégia, não uma proporção obrigatória.

Verificação pergunta se a solução atende às especificações. **Validação** pergunta se ela atende às necessidades reais. É possível implementar corretamente uma especificação que descreve o produto errado.

Manutenção ocupa grande parte do ciclo de vida

Software usado durante anos precisa acompanhar regras, ambientes e necessidades. Além de alterar o código, é necessário entendê-lo, avaliar impacto, testar regressões e disponibilizar mudanças. Isso explica por que manutenção pode representar grande parte do custo total.

Tipo	Objetivo	Exemplo
Corretiva	Corrigir defeito	Corrigir cálculo errado de multa
Adaptativa	Ajustar ao ambiente	Adequar integração a uma nova API externa
Evolutiva	Acrescentar ou melhorar capacidades	Incluir reserva de livros
Preventiva	Reduzir problemas futuros	Reorganizar trecho frágil antes de novas mudanças

Refatoração modifica a estrutura interna preservando o comportamento observável. Extrair um método, renomear uma variável e reorganizar responsabilidades podem ser refatorações. Alterar a fórmula para corrigir o valor entregue ao usuário é correção de comportamento, ainda que a tarefa também inclua refatorações.

Legado não significa irrelevante. Um sistema antigo pode ser essencial e sua substituição, arriscada. O custo de evolução aumenta quando faltam testes, documentação útil e conhecimento do domínio.

Lei de Brooks

Adicionar pessoas a um projeto atrasado pode atrasá-lo ainda mais: quem já trabalha precisa treinar os novos integrantes, cresce a comunicação e certas tarefas não podem ser divididas livremente. A lei descreve uma tendência contextual, não uma proibição de contratar.

Com n pessoas, há $n(n - 1)/2$ pares possíveis de comunicação. Passar de 5 para 8 pessoas aumenta esses pares de 10 para 28. Isso ilustra a coordenação potencial, sem significar que todos os pares precisam conversar o tempo todo.

Pratique antes de continuar

Q03. Todos os 300 testes passaram. Isso prova ausência de bugs? Explique também a diferença entre teste de unidade e integração.

Q04. Uma equipe extrai métodos sem alterar saídas e depois muda uma regra de cálculo. Qual parte é refatoração? Por que adicionar cinco pessoas na última semana pode não resolver seu atraso?

2. Processos: cascata, evolução e agilidade

Um processo define atividades e sua organização para que pessoas construam software juntas. Ele deve ajudar a coordenar decisões, reduzir riscos e produzir qualidade. Não garante sucesso apenas por existir ou por ser seguido nominalmente.

Por que a sequência parecia natural

As engenharias tradicionais já usavam planejamento antecipado, projeto detalhado e execução por etapas. Era natural adaptar essa experiência ao software. No **Waterfall**, requisitos, projeto, implementação, testes e implantação são tratados de forma predominantemente sequencial.

Essa organização oferece marcos claros, documentação e previsibilidade quando as premissas são estáveis. Porém, se o cliente só consegue avaliar a solução tarde, um erro de requisito pode obrigar a refazer projeto, código e testes. Quanto maior a incerteza, mais arriscado é adiar feedback.

Iterativo, incremental e ágil não são sinônimos

Iterativo significa repetir ciclos, refinando a solução. **Incremental** significa acrescentar partes ou capacidades ao produto. Um processo pode combinar ambos. **Ágil** acrescenta valores e práticas de colaboração, adaptação, feedback frequente e entrega de valor.

Os quatro valores do Manifesto Ágil priorizam: indivíduos e interações; software funcionando; colaboração com o cliente; resposta a mudanças. Os itens comparados - processos e ferramentas, documentação abrangente, negociação contratual e seguir um plano - continuam tendo valor. A prioridade relativa não os elimina.

Os princípios incluem entregas frequentes, acolhimento de mudanças, colaboração contínua, ritmo sustentável, simplicidade, excelência técnica e reflexão para melhorar. Agilidade não equivale a improvisação, ausência de projeto ou trabalho sem testes.

Outros modelos dos slides

O **Espiral** organiza ciclos pela definição de objetivos, avaliação de alternativas e riscos, desenvolvimento e testes, e planejamento do próximo ciclo. O **Processo Unificado (UP)** é iterativo e incremental, enfatiza arquitetura e riscos e distribui disciplinas por fases como concepção, elaboração, construção e transição. O **Modelo V** relaciona níveis de especificação a níveis correspondentes de teste.

No enquadramento didático dos slides, UP e Espiral preservam mais planejamento e documentação e usam ciclos mais longos que os métodos ágeis apresentados. Não são cascata pura, pois repetem ciclos. A existência de iteração, isoladamente, não define agilidade; implementações concretas podem variar.

Pratique antes de continuar

Q05. Por que processos sequenciais foram uma escolha historicamente compreensível? Por que podem gerar retrabalho em um produto com necessidades incertas?

Q06. Avalie: “Se o processo tem iterações, ele é ágil; se é ágil, não precisa de documentação”. Explique os dois erros e diferencie UP/Espiral de cascata.

3. XP: qualidade técnica e feedback

Extreme Programming (XP) combina valores, princípios e práticas voltadas ao desenvolvimento de software. Nos slides aparecem comunicação, simplicidade, feedback, coragem, respeito e qualidade do ambiente de trabalho. A ideia é sustentar mudanças com disciplina técnica e colaboração.

Práticas e o problema que enfrentam

Prática	Como ajuda
Testes automatizados	Detectam regressões e permitem feedback repetível
TDD	Faz o comportamento esperado orientar pequenos passos de implementação
Integração contínua	Revela cedo conflitos e incompatibilidades entre mudanças
Refatoração	Reduz deterioração da estrutura sem alterar comportamento
Design simples e incremental	Evita antecipar complexidade sem necessidade
Programação em pares	Combina implementação e revisão contínua, compartilhando conhecimento
Pequenas entregas e cliente próximo	Reduzem a distância entre necessidade e solução

No ciclo básico de **TDD**, escreva um teste que falha pelo comportamento ausente, implemente o necessário para passar e refatore mantendo os testes passando. TDD não substitui todos os testes de integração, sistema ou usabilidade.

Na programação pareada, duas pessoas trabalham juntas, alternando a condução da implementação e a revisão/raciocínio. Benefícios incluem compartilhamento de conhecimento e identificação precoce de problemas. Exige coordenação e ocupa duas pessoas no mesmo trabalho; o retorno depende do contexto. **Mob programming** reúne mais integrantes e pode ajudar em problemas difíceis ou aprendizado coletivo. Revisão por pull request costuma ser assíncrona.

Contratos e capacidade

Escopo fechado fixa uma entrega negociada. Se escopo, prazo e custo ficam rígidos, surpresas podem pressionar a qualidade, sobretudo quando ela é pouco observável. A preferência de XP por **escopo aberto** permite ajustar prioridades mantendo uma capacidade e um custo por período. Exige participação e acompanhamento do cliente; não significa trabalho sem compromisso ou controle.

Uma iteração deve ser curta o suficiente para produzir feedback e longa o suficiente para gerar algo útil. A duração não tem um valor ideal universal. Dependências entre histórias devem ser discutidas: às vezes é possível fatiar o escopo ou simular uma parte; quando não, renegociar a ordem explicando a restrição.

XP E SCRUM SE COMPLEMENTAM: Scrum estrutura o trabalho; XP detalha práticas técnicas. Scrum não obriga uma linguagem, estratégia de testes ou programação pareada. XP não é “programar o mais rápido possível sem planejar”.

Pratique antes de continuar

Q07. Um time usa Scrum, mas integra código apenas na véspera da entrega e quase não testa. Cite três práticas de XP e explique que risco cada uma reduz.

Q08. Compare contratos de escopo aberto e fechado. Por que fixar escopo, prazo e custo pode pressionar a qualidade? Cite uma vantagem e um custo da programação em pares.

4. Scrum: estrutura e responsabilidades

Scrum é um framework leve para gerar valor por soluções adaptativas em problemas complexos. É propositalmente incompleto: define responsabilidades, eventos e artefatos, mas depende de técnicas complementares para arquitetura, programação, testes e implantação.

Sua base empírica combina **transparência**, para tornar o trabalho e os critérios compreensíveis; **inspeção**, para examinar produto e progresso; e **adaptação**, para agir sobre o que foi aprendido. Inspecionar sem adaptar transforma feedback em ritual sem efeito.

Os valores do Guia 2020 são compromisso, foco, abertura, respeito e coragem. A Scrum Team é multifuncional e autogerenciável: reúne competências para entregar valor e decide internamente como organizar o trabalho. O Guia descreve times tipicamente com 10 pessoas ou menos.

Quem responde por quê

Product Owner (PO). Responde pela maximização do valor e pela gestão eficaz do Product Backlog: objetivo do produto, clareza, ordenação e transparência dos itens. Pode delegar atividades, mas mantém a responsabilidade. É uma pessoa, não um comitê. Deve compreender o domínio e conseguir tomar decisões com os stakeholders.

Developers. Criam o plano do Sprint, produzem o incremento, cumprem a Definition of Done e adaptam o plano diariamente. “Developer” aqui não significa somente programador: pode incluir diferentes competências necessárias ao trabalho.

Scrum Master (SM). Ajuda a estabelecer Scrum e a melhorar a eficácia da equipe. Orienta autogerenciamento, promove a remoção de impedimentos e ajuda a tornar os eventos produtivos. Não é chefe nem distribuidor obrigatório de tarefas. Não precisa resolver pessoalmente todo obstáculo técnico.

Exemplo aplicado

Uma nova regra exige alterar uma transferência. O PO esclarece valor, impacto e prioridade. Os Developers analisam implementação, testes e capacidade. O SM ajuda com impedimentos e aplicação do processo. Um stakeholder pode solicitar uma mudança, mas a solicitação não se torna automaticamente a prioridade máxima do Sprint.

POs problemáticos incluem o indisponível, que não esclarece necessidades; o sem poder de decisão, que depende sempre de terceiros; e o que atua como chefe técnico, impondo execução e comprometendo autonomia. Também há problema quando se aceitam todas as demandas sem ordenar valor e capacidade.

Pratique antes de continuar

Q09. Um diretor exige que o PO atribua tarefas técnicas e obrigue o time a assumir mais trabalho do que considera viável. Analise a situação e distribua as responsabilidades corretamente.

Q10. O banco de testes caiu e uma história está pouco clara. Como PO, Developers e SM podem atuar? Relacione a resposta aos três pilares do Scrum.

4.1 Eventos: cada conversa tem uma finalidade

O **Sprint** é o evento que contém os demais. Tem duração fixa de um mês ou menos. Um novo Sprint começa imediatamente após o anterior. A duração de 15 dias usada nos exercícios é um exemplo, não uma regra universal.

Evento	Propósito	Limite no Guia 2020
Sprint Planning	Definir por que, o que e como trabalhar	Até 8 h para Sprint de 1 mês
Daily Scrum	Inspecionar avanço e adaptar plano	15 min por dia útil
Sprint Review	Inspecionar resultado e adaptar próximos passos	Até 4 h para Sprint de 1 mês
Sprint Retrospective	Melhorar qualidade e eficácia do trabalho	Até 3 h para Sprint de 1 mês

Para Sprints menores, Planning, Review e Retrospective normalmente são mais curtas. Daily continua sendo um evento de 15 minutos.

Planning e Daily

No Planning, o time define um **Sprint Goal** que explica por que o Sprint é valioso. Os Developers selecionam itens em discussão com o PO e planejam como entregá-los. A seleção considera capacidade, experiência e DoD; não é uma imposição unilateral do PO.

A Daily é para os Developers coordenarem seu trabalho rumo ao objetivo. Não é prestação de contas ao chefe. As perguntas “o que fiz, o que farei e quais impedimentos tenho” podem ajudar, mas o Guia 2020 não obriga esse roteiro. Discussões técnicas longas podem ocorrer depois com os envolvidos.

Review e Retrospectiva

Na **Review**, time e stakeholders inspecionam o resultado e o contexto, discutindo adaptações do produto e do backlog. É uma sessão de trabalho, não só demonstração nem portão obrigatório para liberar software.

Na **Retrospectiva**, o time examina colaboração, práticas, ferramentas, qualidade e dificuldades para decidir melhorias. Um exemplo é incorporar à DoD a atualização da documentação de API, se o problema recorrente são interfaces modificadas sem documentação correspondente.

Mudança e cancelamento

Durante o Sprint, não se deve comprometer o Sprint Goal nem reduzir qualidade. O escopo pode ser esclarecido e renegociado com o PO conforme se aprende. **Apenas o PO pode cancelar o Sprint, quando o objetivo se torna obsoleto.** Um atraso isolado não é o critério descrito pelo Guia.

Pratique antes de continuar

Q11. Diferencie Review de Retrospectiva com um exemplo de decisão em cada uma. A Daily exige as três perguntas tradicionais? Justifique.

Q12. Uma mudança de mercado torna inútil o objetivo do Sprint. Quem pode cancelá-lo? Compare com uma situação em que apenas uma tarefa atrasou.

4.2 Artefatos, qualidade e estimativas

Artefato	O que representa	Compromisso
Product Backlog	Lista ordenada e emergente de trabalho do produto	Product Goal
Sprint Backlog	Objetivo, itens selecionados e plano de execução	Sprint Goal
Incremento	Resultado utilizável que se soma aos anteriores	Definition of Done

O Product Backlog pode conter histórias, defeitos, melhorias técnicas e investigações. Itens próximos da execução tendem a estar mais esclarecidos e menores. Os distantes podem ser amplos, pois detalhar cedo demais cria trabalho que pode perder utilidade. Refinamento é uma atividade contínua, não um evento formal adicional obrigatório do Scrum.

O Sprint Backlog não é apenas uma lista de tarefas: inclui o **porquê, o quê e o como**. É um plano dos Developers, adaptado à medida que aprendem. O incremento precisa ser utilizável e atender à DoD; “quase pronto” não é pronto.

Definition of Done e aceitação

Crítérios de aceitação descrevem resultados esperados de uma história específica: ao cancelar uma reserva futura, o horário deve voltar a ficar disponível. **DoD** descreve medidas de qualidade para o incremento: testes relevantes passando, revisão e documentação necessária atualizada, por exemplo.

Os exemplos dos slides com 75% ou 80% de cobertura não são percentuais oficiais obrigatórios. Cobertura é um indicador de execução, não prova de qualidade dos testes. Pelo Guia, um item que não atende à DoD não integra o incremento e retorna ao Product Backlog para consideração futura.

Story Points e velocidade

Story Points expressam tamanho relativo, levando em conta esforço, complexidade e incerteza conforme a referência do time. Escalas como 1, 2, 3, 5, 8 e 13 evitam uma precisão artificial. Em Planning Poker, estimativas são reveladas e diferenças levam à discussão do entendimento.

Velocidade é a quantidade de pontos concluídos por Sprint segundo a prática adotada. Serve ao planejamento do próprio time em contexto comparável. Não é um índice universal de produtividade. Times com 24 e 16 pontos podem ter escalas, produtos e critérios distintos: não se conclui que o primeiro é 50% mais produtivo.

AJUSTE DO MATERIAL: a aula 5 contém uma questão que menciona dois papéis e dois artefatos. Pelo Guia 2020 incluído no ZIP, são três responsabilidades, três artefatos e cinco eventos contando o Sprint. Histórias e Story Points não são formatos obrigatórios do framework.

Pratique antes de continuar

Q13. Uma história funciona no computador do autor, mas falta revisão exigida pela DoD. Ela faz parte do incremento? Diferencie isso de um critério de aceitação não atendido.

Q14. Dois times entregam 30 e 20 pontos por Sprint. O primeiro é 50% mais produtivo? Cite dois usos adequados de estimativas e explique o conteúdo do Sprint Backlog.

5. Kanban: fluxo, limites e gargalos

Kanban torna visível o fluxo de trabalho e ajuda a gerenciá-lo. Sua lógica **pull** permite puxar trabalho quando existe capacidade, em vez de empurrar tarefas para etapas já ocupadas. Um quadro sem políticas e sem gestão de fluxo não representa sozinho toda a abordagem.

Entenda o que o limite está contando

O **WIP (Work in Progress)** representa itens em andamento dentro dos limites observados. O **limite de WIP** é a quantidade máxima acordada para uma etapa ou parte do fluxo. Ele restringe trabalho simultâneo, torna gargalos visíveis e favorece terminar antes de começar mais.

No quadro usado na aula 4, uma etapa intermediária tem subcolunas “em execução” e “concluída, aguardando a próxima”. Ambas ocupam sua capacidade. Na última etapa, itens efetivamente finalizados saem do WIP. Na especificação daquele exemplo, grupos de tarefas na mesma linha representam uma história; não se deve contar cada tarefa como uma nova história.

Exemplo resolvido: tudo parece parado

Imagine especificação no limite 2, implementação no limite 5 e validação no limite 3. Existem tarefas implementadas esperando validação e histórias especificadas esperando implementação. Abrir outra especificação aumenta estoque, mas não a entrega.

A intervenção é observar de trás para frente: colaborar para concluir validações, liberando capacidade para puxar itens implementados. Isso libera implementação e, depois, especificação. A colaboração respeita competências e critérios de qualidade; não significa aprovar sem testar.

O objetivo é melhorar o fluxo do sistema, não manter cada pessoa sempre ocupada iniciando tarefas.

Aumentar limites pode ser uma decisão experimental justificada, mas não deve ser reação automática para esconder um bloqueio.

Comparação com Scrum

Kanban não exige Sprints, PO, Scrum Master ou os eventos formais do Scrum. O time define políticas e cadências adequadas ao contexto. Pode ser combinado com Scrum: o Sprint continua existindo, enquanto limites ajudam a controlar trabalho dentro dele.

Um cartão pode retornar por reprovação ou retrabalho, conforme as políticas do quadro. O importante é tornar esse trabalho visível e entender sua causa, não fingir que ele deixou de existir. Colunas de revisão e validação, critérios explícitos e WIP ajudam a evitar a pressa por transferir trabalho incompleto.

Pratique antes de continuar

Q15. Especificação e implementação estão cheias; há tarefas implementadas esperando uma validação também cheia. O que priorizar? Explique por que aumentar o WIP não resolve necessariamente.

Q16. Uma coluna possui 3 itens em execução e 2 concluídos aguardando a etapa seguinte, com limite 4. Há violação? Um cartão pode voltar no quadro? Justifique.

5.1 Métricas de fluxo: cálculo e interpretação

Throughput é a quantidade de itens concluídos por unidade de tempo. **Lead Time** é o tempo decorrido entre entrada e saída dos limites definidos para a medição, incluindo espera. **Cycle Time** costuma considerar do início do trabalho à conclusão; como convenções variam, declare os marcos de início e fim.

Lei de Little

Em condições estáveis, com médias e limites de medição coerentes:

$$\text{WIP médio} = \text{Throughput médio} \times \text{Lead Time médio}$$

As unidades confirmam o raciocínio: tarefas/dia $\times$ dias = tarefas. Também podemos calcular $\text{Lead Time} = \text{WIP} / \text{Throughput}$ ou $\text{Throughput} = \text{WIP} / \text{Lead Time}$. A relação não permite ignorar instabilidade, perdas de itens ou fronteiras de medição diferentes.

Exemplo resolvido do questionário

O processo possui tempos médios de 4, 3, 10 e 5 dias. A etapa C é a mais lenta, com vazão de 0,5 tarefa por dia. Pelo enunciado, assuma fluxo estável e a mesma vazão efetiva ao longo da sequência.

Etapa	Conta	WIP médio
A	$0,5 \times 4$	2 tarefas
B	$0,5 \times 3$	1,5 tarefa
C	$0,5 \times 10$	5 tarefas
D	$0,5 \times 5$	2,5 tarefas
Total	$0,5 \times 22$	11 tarefas

O resultado fracionário é uma **média ao longo do tempo**, não a existência literal de meia tarefa em um instante. Também não é automaticamente o limite de WIP que deveria ser configurado.

O que a relação ensina

Se a vazão permanece em 2 itens/dia e o WIP médio sobe de 8 para 12, o tempo médio passa de 4 para 6 dias. Mais trabalho iniciado pode gerar mais espera, sem aumentar entregas. Reduzir WIP pode ajudar o fluxo, mas um limite excessivamente baixo também pode deixar capacidade sem alimentação: é preciso observar e ajustar.

Para responder exercícios: defina o fluxo, escreva a fórmula, converta unidades, substitua valores e interprete. Não multiplique “tarefas por semana” por “dias” sem converter. O gargalo limita a vazão sustentável, mas a hipótese de mesma vazão precisa fazer sentido para o processo dado.

Pratique antes de continuar

Q17. Um fluxo estável conclui 3 tarefas/dia e tem Lead Time médio de 4 dias. Calcule WIP. Se WIP chegar a 18 com vazão constante, qual será o novo Lead Time?

Q18. Uma equipe entrega 10 itens/semana e mantém WIP médio 15. Qual é o Lead Time em semanas? Por que esse valor não determina, sozinho, o limite ideal do quadro?

6. Requisitos e histórias de usuário

Engenharia de Requisitos envolve compreender o domínio e as necessidades, analisar conflitos, especificar o que foi acordado e validar o entendimento. Requisitos de diferentes interessados não se tornam consistentes automaticamente: pessoas podem desejar resultados incompatíveis.

Um **stakeholder** influencia, interessa-se pelo produto ou é afetado por ele. Usuários, patrocinadores, suporte, gestores e reguladores podem ter perspectivas relevantes. Ouvir o cliente não elimina a necessidade de observar contexto e validar hipóteses.

Uma história registra valor, não só trabalho

Formato útil: **Como [papel], quero [objetivo], para [benefício]**. Exemplo: “Como estudante, quero renovar um empréstimo, para continuar usando o livro sem precisar ir à biblioteca.” O papel deve representar quem tem a necessidade, e o benefício explica por que a funcionalidade importa.

Os **3 Cs** são: **Cartão**, lembrete resumido; **Conversa**, esclarecimento das regras e alternativas; **Confirmação**, critérios que permitem verificar se a necessidade foi atendida. Uma frase isolada não substitui a conversa sobre regras de negócio.

INVEST

Critério	Pergunta prática
Independente	Consigo reduzir dependências de outras histórias?
Negociável	Há espaço para discutir a solução?
Valiosa	Existe benefício para usuário ou negócio?
Estimável	Entendemos o suficiente para estimar?
Pequena	Cabe em uma entrega ou iteração administrável?
Testável	Conseguimos verificar o resultado?

Independência é um objetivo de fatiamento, não a afirmação de que dependências nunca existem. Para “editar arquivo” antes de “abrir arquivo”, uma opção é testar edição sobre conteúdo inicial fornecido. Se a dependência for essencial, equipe e PO precisam negociar de forma explícita.

Épicos e trabalho técnico

Um **épico** é uma necessidade grande que precisa ser dividida: “aplicar provas online” pode virar criação de prova, realização pelo aluno, correção e consulta de resultados. O fatiamento deve preservar valor observável, evitando apenas “fazer banco”, “fazer tela” e “fazer API” como supostas histórias independentes de usuário.

Tarefas técnicas são legítimas: configurar ambiente, corrigir defeitos, refatorar e testar. **Spikes** são investigações para reduzir incerteza, preferencialmente com objetivo, duração e resultado esperado. Não precisam ser disfarçados como histórias de usuário.

Pratique antes de continuar

Q19. Escreva duas histórias para estudantes e uma para bibliotecários, cada uma com papel, objetivo e benefício.

Q20. Divida “Como professor, quero gerenciar toda a avaliação online da turma” em três histórias menores com valor. Diferencie uma delas de uma tarefa técnica e de um spike.

6.1 Aceitação, qualidade e erros de escrita

CrITÉRIOS de aceitação precisam ser observáveis. Para “Como aluno, quero cancelar minha reserva futura”, exemplos são: a reserva deixa de estar ativa; o horário fica disponível; outra pessoa não consegue cancelar uma reserva que não lhe pertence; reservas passadas não podem ser canceladas por esse fluxo.

Uma estrutura possível é **Dado um contexto, quando ocorre uma ação, então ocorre um resultado**. Exemplo: “Dada uma reserva futura do aluno autenticado, quando ele confirma o cancelamento, então a reserva fica cancelada e o horário volta a ficar disponível.” Não é um formato obrigatório, mas torna a condição verificável.

Não confunda dois níveis de critério

Critério de aceitação	Definition of Done
Descreve comportamento específico da história	Descreve qualidade necessária ao incremento
Ex.: reserva cancelada libera horário	Ex.: testes relevantes e revisão concluídos
Varia conforme a necessidade	Pode valer para muitas histórias

A distinção é didática: critérios de aceitação também podem incluir uma restrição de qualidade específica. Requisitos não funcionais podem aparecer no backlog, nos critérios pertinentes e na DoD quando forem transversais. Não desaparecem por se adotar histórias.

Anti-patterns cobrados

Grande demais: “gerenciar toda a universidade”. Dificulta estimar, testar e concluir. **Vaga:** “ser intuitivo”. Não especifica resultado verificável. **Sem valor:** registra uma ação sem esclarecer sua utilidade. **Tarefa técnica disfarçada:** “Como desenvolvedor, quero criar uma tabela”, sem explicar necessidade do produto. **Sem confirmação:** abre espaço para divergência sobre conclusão. **Solução prematura:** “quero um botão azul no canto”, antes de entender a necessidade.

Nem todo detalhe visual é inadequado: se houver uma necessidade real de identidade ou acessibilidade, ele pode ser relevante. O erro é fixar a implementação sem justificativa e sem esclarecer o problema.

Como validar requisitos

Verifique clareza, consistência, viabilidade e correspondência às necessidades. Um protótipo pode revelar mal-entendidos, mas não substitui regras que não aparecem na tela. **Dogfooding** significa usar o próprio produto para aprender sobre ele; ajuda a perceber problemas, mas não substitui usuários representativos quando o contexto da equipe é diferente.

Pratique antes de continuar

Q21. Reescreva “Como usuário, quero um sistema moderno com banco novo”. Identifique dois problemas do enunciado e proponha uma história com valor explícito.

Q22. Para “fechar um formulário para novas respostas”, escreva três critérios de aceitação e dois exemplos de DoD. Não confunda comportamento da funcionalidade com processo de qualidade.

6.2 Casos de uso e escolha da técnica

Um **caso de uso** descreve como um ator alcança um objetivo interagindo com o sistema. Inclui um fluxo normal e extensões que tratam alternativas, erros ou detalhamentos. Pode registrar precondições e resultados esperados. O ator é um papel externo: pessoa, organização ou outro sistema, conforme o contexto.

Casos de uso não são algoritmos internos nem roteiros de botões e cores. O foco é o comportamento exigido, não a implementação. Diagramas ajudam a mostrar atores e objetivos, mas não substituem o detalhamento textual quando as regras são importantes.

Exemplo resolvido: reservar livro

Ator: estudante. **Precondição:** estudante identificado e habilitado conforme as regras da biblioteca.

1. Estudante pesquisa um título. 2. Sistema apresenta os títulos encontrados e sua situação. 3. Estudante seleciona um título e solicita reserva. 4. Sistema verifica elegibilidade e registra a reserva. 5. Sistema confirma a reserva e informa as condições de retirada.

Extensão 2a: nenhum título é encontrado; o sistema informa a ausência de resultados e permite nova pesquisa. **Extensão 4a:** estudante atingiu o limite de reservas; o sistema informa o motivo e não cria outra.

Extensão 4b: já existe uma reserva ativa do mesmo estudante para o título; o sistema informa a situação e evita duplicação.

A boa extensão aponta **passo + condição + comportamento esperado**. Apenas escrever “pode dar erro” não é suficiente. Alternativas válidas, como outra forma de pagamento, também podem ser extensões.

Quando usar cada abordagem

Contexto	Técnica especialmente útil	Justificativa
Produto com melhorias frequentes	Histórias	Facilitam pequenas entregas e repriorização
Fluxo com muitas regras e exceções	Casos de uso	Explicitam interações e alternativas
Dúvida sobre demanda e proposta de valor	MVP	Produz aprendizado com usuários reais
Dúvida sobre navegação e entendimento	Protótipo	Permite observar uso antes de implementar

Essas técnicas podem coexistir. Uma história prioriza a necessidade, um caso de uso detalha o fluxo, um protótipo torna a proposta testável e critérios de aceitação definem resultados. Não é necessário escolher uma técnica para excluir todas as outras.

Pratique antes de continuar

Q23. No caso “Comprar livro”, há pesquisa, carrinho, endereço, entrega e pagamento. Escreva quatro extensões, identificando etapa, condição e resposta do sistema.

Q24. Escolha e justifique uma técnica para: a) evolução semanal de um delivery; b) matrícula com pré-requisitos e conflitos; c) serviço novo cuja demanda é desconhecida. Explique por que podem ser combinadas.

7. MVP: validar antes de ampliar o investimento

Um **Produto Mínimo Viável** é um experimento construído com o mínimo necessário para aprender sobre uma hipótese relevante do produto ou negócio. Seu valor está em reduzir incerteza com evidências, e não em possuir o menor número possível de telas.

Comece pela hipótese

Para um serviço de professores particulares, a hipótese pode ser: “Estudantes solicitarão uma aula paga com professores verificados”. A equipe pode divulgar uma oferta simples, receber solicitações e organizar encontros manualmente. Não precisa construir de imediato busca sofisticada, pagamentos integrados e recomendação automática.

Defina antes: público, hipótese, experiência oferecida, evidência a observar e critério de decisão. Pedidos reais, pagamentos ou retornos recorrentes dizem algo diferente de elogios à ideia. Uma landing page com cadastro de interesse mede interesse inicial; não prova, sozinha, disposição a pagar ou retenção.

MVP, protótipo e incremento

Protótipo: representação para explorar ou validar uma proposta, principalmente interação no contexto das aulas 8 e 9. **Incremento:** parte utilizável do produto integrada ao que existia e que atende à qualidade exigida. **MVP:** experimento orientado a aprendizado sobre hipótese do produto.

Eles podem se sobrepor. O primeiro incremento pode servir de MVP se foi escolhido para testar uma hipótese relevante. Porém, não se torna MVP apenas por ser a primeira versão. Um protótipo em papel pode revelar usabilidade sem demonstrar demanda de mercado.

Aprender, persistir ou pivotar

Após observar resultados, a equipe pode ajustar e repetir, mudar a direção, interromper ou ampliar a solução.

Pivô é uma mudança significativa baseada no aprendizado: focar outro segmento, transformar uma funcionalidade em produto (zoom-in), passar de aplicação para plataforma ou trocar uma abordagem tecnológica.

Uma mudança de cor não costuma ser pivô. E uma troca de estratégia comercial não prova, por si só, que ocorreu um ciclo de experimentação; justifique a relação entre evidência e mudança.

Não confunda mínimo com descuido. O experimento deve ser suficientemente confiável para testar a hipótese. Em contextos críticos, falhas podem inviabilizar experimentos diretos; simulação e ambientes controlados podem ser necessários. A incerteza a testar determina o formato, não uma regra universal de “duas semanas”.

Pratique antes de continuar

Q25. Proponha um MVP sem desenvolver aplicativo para organizar caronas universitárias. Defina hipótese, operação mínima, métrica e decisão possível.

Q26. A primeira Sprint entregou cadastro e login. Isso é necessariamente um MVP? Diferencie um pivô de uma melhoria pequena e dê um exemplo de cada.

7.1 Testes A/B: medir uma diferença

Um teste A/B compara uma versão de controle A com uma alternativa B. A alocação aleatória e grupos comparáveis ajudam a atribuir a diferença à alteração estudada. Defina a unidade de análise, mantenha a atribuição consistente e planeje a métrica e o tamanho de amostra antes de decidir.

Métrica bem definida

Para comparar “Adicionar ao carrinho” e “Compre já”, uma conversão possível é a adição de produto ao carrinho. **Taxa = usuários únicos que adicionaram ao carrinho / usuários únicos expostos à versão.** Conte cada usuário conforme a regra planejada, sem misturar número de eventos no numerador e pessoas no denominador.

Se o objetivo for compra concluída, o evento deve ser a compra, e a análise muda. Uma melhoria em cliques não demonstra automaticamente melhoria em receita. A escolha precisa corresponder à hipótese.

Exemplo resolvido: 5% para 6%

Ganho absoluto: $6\% - 5\% = 1$ ponto percentual. Ganho relativo: $(6\% - 5\%)/5\% = 20\%$. Se a calculadora pedir efeito relativo, informe 20%, não 1%.

O questionário pede confiança de 95%, poder de 80%, teste bicaudal e divisão 50/50. A resposta registrada apresenta **8.158 pessoas por versão e 16.316 no total**, aproximadamente 8,2 mil por grupo. Esse número é uma referência do arquivo, não uma constante: métodos de cálculo e correções podem produzir pequenas diferenças.

Confiança de 95%, na interpretação frequencista, relaciona-se à cobertura do procedimento em repetições; não significa “95% de chance de B ser melhor”. **Poder de 80%** é a probabilidade planejada de detectar o efeito definido quando ele existe, sob as hipóteses do cálculo. **Bicaudal** considera diferenças em ambos os sentidos.

Efeitos menores geralmente exigem mais participantes. Aumentar poder ou exigir maior confiança também tende a exigir mais dados. Não conclua que “não há efeito” apenas porque o resultado não foi significativo: pode faltar informação. Evite encerrar um teste convencional assim que surge um resultado favorável, sem uma regra de análise apropriada.

ESCOPO DA PROVA: o questionário exige configurar uma calculadora, não derivar a fórmula estatística da amostra. Domine métrica, ganho absoluto/relativo, parâmetros, amostra por versão e interpretação.

Pratique antes de continuar

Q27. A converteu 100 de 2.000 usuários e B converteu 120 de 2.000. Calcule taxas, ganho absoluto e relativo. Esses números, sozinhos, permitem declarar superioridade estatística?

Q28. Uma equipe conta 300 cliques de 100 pessoas como 300 conversões em 100 usuários e termina o teste no primeiro dia favorável. Identifique os problemas e proponha correções.

8. Prototipação: escolher o que representar

Prototipar torna uma ideia observável antes da implementação definitiva. Ajuda a comparar alternativas, comunicar propostas e descobrir problemas quando a mudança ainda é barata. O protótipo precisa responder a uma pergunta; não precisa reproduzir o produto inteiro nem conter dados reais.

Artefatos e fidelidade

Artefato	O que comunica	Melhor aplicação
Sketch	Ideia desenhada rapidamente	Explorar alternativas iniciais
Wireframe	Estrutura e organização	Posicionar conteúdo e navegação
Mockup	Aparência visual	Avaliar cores, tipografia e ícones
Protótipo interativo	Fluxo e comportamento simulados	Observar execução de tarefas

Um protótipo pode usar sketches, wireframes ou mockups como base. Papel também pode ser interativo: o participante indica uma ação e o operador troca a folha. Não existe uma sequência obrigatória em que todo projeto precise produzir os quatro artefatos.

Baixa fidelidade favorece conceito, organização e fluxo com baixo custo de alteração. **Média fidelidade** detalha navegação e comportamento. **Alta fidelidade** aproxima conteúdo, aparência e interações do produto final. Maior fidelidade pode consumir tempo, criar apego e desviar atenção para estética antes de resolver o fluxo.

Exemplo resolvido

Uma equipe quer testar um aplicativo de despesas de viagem. A pergunta é se as pessoas entendem criar viagem, registrar despesa, dividir valor e consultar dívidas. Basta representar esse fluxo e dados ilustrativos. Autenticação, banco e notificações podem ser simulados se não forem o objeto da investigação.

A escolha se justifica por **aprendizado, redução de risco, menor custo de mudança e escopo coerente**. Dizer apenas “é mais rápido” deixa de explicar por que economizar nesse momento é útil.

Antes de desenhar telas, represente o fluxo: objetivo, etapas, decisões e estados de erro relevantes. Uma coleção de telas bonitas pode não formar uma experiência compreensível. Histórias e critérios de aceitação ajudam a escolher o que o protótipo precisa mostrar.

Pratique antes de continuar

Q29. Escolha artefatos para: a) comparar rapidamente organizações de tela; b) avaliar tipografia e cores; c) verificar se alguém conclui um agendamento. Justifique cada escolha.

Q30. Uma startup propõe construir banco, login e notificações antes de testar o fluxo principal. Compare com prototipar apenas o fluxo. Discuta risco, custo, aprendizado e escopo.

8.1 Planejar e conduzir testes de protótipo

Testar um protótipo é observar pessoas representativas tentando realizar tarefas. Apresentar a solução e explicar onde clicar produz outro tipo de interação: uma demonstração. Se a pergunta é sobre descoberta e compreensão, ensinar o caminho contamina a observação.

Um plano coerente de teste

Objetivo: verificar se alunos solicitam revisão de nota sem ajuda. **Participantes:** alunos com características e contexto pertinentes, que não participaram da criação. **Cenário:** “Você recebeu uma nota que considera incorreta e precisa pedir revisão antes do prazo”. **Tarefa:** “Envie uma solicitação de revisão para essa disciplina”.

Evidências: conclusão independente, tempo, erros, hesitações, retornos, ajuda e entendimento da confirmação. **Decisão:** investigar e corrigir barreiras recorrentes ou graves e repetir o teste. Um limiar como 80% de sucesso pode ser definido para aquele exercício, mas não é uma norma universal nem prova estatística com cinco pessoas.

Participantes e papéis

Priorize o público-alvo: enfermeiros para um sistema de rotina de enfermagem, por exemplo. Desenvolvedores e colegas podem ajudar em verificações preliminares, mas sua familiaridade pode esconder dificuldades do usuário real. Mais pessoas de perfil inadequado não necessariamente oferecem evidência mais útil que um grupo menor representativo.

O **facilitador** apresenta contexto e tarefas; o **observador** registra comportamento; o **operador** simula telas quando necessário; o **participante** tenta realizar a tarefa. É possível acumular papéis em equipes pequenas, sem abandonar o registro.

Perguntas neutras e pensar em voz alta

Prefira “O que você esperava encontrar?” a “Você não acha que deveria clicar aqui?”. Pedir que a pessoa pense em voz alta pode revelar expectativas, mas não deve virar uma interrupção constante. Explique que o protótipo está sendo testado e que certas partes serão simuladas; isso é diferente de ensinar a solução.

Evite apontar, olhar insistentemente para o botão correto, acenar confirmando escolhas ou justificar decisões. Se for necessário ajudar para prosseguir, registre a ajuda e não conte o resultado como sucesso autônomo.

Pratique antes de continuar

Q31. Escreva objetivo, cenário e tarefa para testar cancelamento de uma reserva. Não cite botões nem o caminho da interface. Liste quatro evidências.

Q32. Um facilitador mostra todas as telas, indica o calendário e registra sucesso quando a pessoa termina. Identifique três problemas e como corrigi-los.

8.2 Interpretar resultados e decidir melhorias

Conclusão da tarefa é importante, mas não é a única evidência. Uma pessoa pode concluir após vários erros, pedir ajuda e continuar sem entender se a operação ocorreu. Elogios também podem coexistir com dificuldade observada.

Opinião, comportamento e inferência

Opinião: “não gostei da cor”. **Comportamento:** “procurou a função no menu Disciplinas três vezes”.

Inferência: “o rótulo ou a organização talvez não corresponda à expectativa”. **Ação:** revisar a navegação e realizar nova rodada para avaliar a hipótese.

Separe essas quatro etapas para não transformar uma suposição em fato. O usuário conhece sua experiência, mas não necessariamente a melhor solução de design. Ao sugerir outro botão, pode estar revelando falta de visibilidade do atual.

Exemplo resolvido de interpretação

Cinco pessoas testam uma tarefa. Uma não conclui; três mostram dificuldade de localização; apenas uma não tem dificuldade registrada. Mesmo que várias cheguem ao fim, há indício de problema recorrente. A equipe deve examinar onde procuraram a função e testar uma reorganização, não afirmar que a interface é clara apenas pelo número de conclusões.

Na questão 2 do questionário 3 há uma ambiguidade: a tabela diz que P4 encontrou “Matrículas” depois de três tentativas, enquanto a conclusão destaca três participantes. Não é seguro inventar a conclusão final de P4; registre a ambiguidade. É seguro afirmar que **P1, P2, P4 e P5 apresentam dificuldades ou expectativas divergentes** e que só P3 não tem dificuldade registrada.

O que um protótipo não demonstra

Um fluxo clicável pode ajudar a avaliar entendimento de uma mensagem, mas não comprova desempenho real, segurança de implementação ou robustez de uma integração. Alguns critérios podem ser simulados; outros precisam de testes técnicos posteriores. Registre o que foi efetivamente testado.

A sequência de aprendizado é criar, testar, observar, interpretar, modificar e testar novamente. Não é necessário obedecer a toda sugestão nem defender a primeira solução. O objetivo é reduzir incerteza e orientar implementação.

Pratique antes de continuar

Q33. Quatro de cinco pessoas terminam uma tarefa; três delas pediram ajuda. A equipe declara 80% de sucesso e aprova a interface. Analise a conclusão e proponha novos registros.

Q34. Para testar registro de medicamentos, estão disponíveis cinco desenvolvedores, dez administrativos e quatro enfermeiros externos à criação. Quem priorizar? Os outros grupos são inúteis?

Q35. Um usuário pede um botão novo. Como investigar a necessidade por trás da sugestão? Por que alta fidelidade não garante validade do teste?

9. Propriedades de um bom projeto

Projeto de software divide o problema em módulos e define suas relações. Um módulo pode ser função, classe, pacote, biblioteca ou serviço. Possui uma **interface**, pela qual é usado, e uma **implementação**, que realiza o trabalho. A divisão deve reduzir o esforço de compreender e modificar o sistema.

Integridade conceitual

É a coerência das decisões ao longo do sistema. Nomes seguem convenções, operações similares se comportam de forma consistente e a interface não exige reaprender a mesma ação em cada tela. Misturar arbitrariamente padrões de nome e formatos aumenta a carga de compreensão.

Consistência não exige aplicar uma técnica inadequada a todos os problemas. Mudanças de padrão podem ser justificadas pelo contexto, mas precisam de intenção e clareza.

Ocultamento de informação

O módulo deve esconder decisões internas, principalmente as suscetíveis a mudança. Se clientes manipulam diretamente a coleção de veículos de um estacionamento, uma troca de estrutura de dados afeta todos eles. Operações como entrar(), sair() e consultar() tornam a implementação substituível.

Declarar um atributo como private ajuda, mas não basta se um getter devolve uma estrutura mutável que permite alterar livremente o estado. A interface precisa proteger regras e não vaziar detalhes desnecessários.

Outro exemplo dos slides: devolver um **ResultSet** expõe detalhes de banco; devolver um **Book** pode oferecer uma abstração mais adequada ao cliente. A escolha depende do propósito da interface, mas a pergunta central é quem precisa conhecer a tecnologia interna.

Coesão e acoplamento

Coesão mede a relação entre responsabilidades dentro de um módulo. Uma pilha com push, pop e size possui operações diferentes, mas voltadas ao mesmo conceito. Responsabilidade única não exige um único método.

Acoplamento descreve dependências entre módulos. A qualidade importa tanto quanto a quantidade: depender de uma interface pública estável pode ser adequado. Depender de detalhes internos ou de um formato compartilhado implicitamente torna a mudança mais frágil.

Um módulo com interface pequena que esconde trabalho substancial pode simplificar o sistema. Um conjunto de muitos módulos que quase apenas repassam chamadas pode aumentar a complexidade de navegação sem esconder detalhes úteis.

Pratique antes de continuar

Q36. Uma classe expõe publicamente um mapa de veículos. Qual propriedade é prejudicada? Proponha uma interface melhor e explique o ganho se o armazenamento mudar.

Q37. Explique por que uma classe com vários métodos pode ter alta coesão. Dê um exemplo de falta de integridade conceitual no código ou na interface.

9.1 Dependências: qualidade e equilíbrio

Acoplamento estrutural aceitável e ruim

Há dependência estrutural quando uma classe usa tipos, chamadas, parâmetros ou herança de outra, conforme a análise adotada. **Aceitável:** um serviço usa uma interface estável de pagamento e conhece apenas as operações necessárias. **Ruim:** acessa atributos internos, detalhes do banco ou inúmeras operações instáveis de outro módulo.

Não é obrigatório criar uma interface nova para toda classe. Depender de uma classe concreta estável de biblioteca pode ser adequado. A regra é reduzir exposição desnecessária e avaliar custo da abstração.

Dependência sem referência explícita

Se A escreve um arquivo com “nome;cpf,email” e B o lê assumindo essa ordem, uma mudança de A pode quebrar B mesmo sem importações ou chamadas. O material chama atenção para esse **acoplamento evolutivo**: alterações em um componente repercutem no outro.

Uma melhoria é estabelecer um contrato explícito, encapsular acesso por um serviço ou controlar a evolução do formato. Apenas mover os arquivos de lugar não elimina a dependência.

Acoplamento temporal e estado global

Existe acoplamento temporal quando uma operação só pode ser chamada depois de outra. Abrir um arquivo antes de lê-lo pode ser uma sequência natural. Já exigir `setRadius()` antes de `getArea()`, quando o objeto poderia nascer válido, pode ser evitado com `new Circle(5)`.

Uma variável global que vários métodos alteram cria dependências difíceis de rastrear. Em “`g = 1; f(); imprimir(g)`”, não é possível determinar a saída sem conhecer efeitos de `f`. Encapsular estado e tornar dependências explícitas facilita raciocínio e testes.

Classitis e o limite de “dividir para conquistar”

Fragmentar uma funcionalidade em dezenas de pequenas classes pode obrigar o leitor a atravessar muitas interfaces para entender uma ação. Maximizar coesão isoladamente não minimiza complexidade global. Por outro lado, concentrar tudo num `main` reduz divisões, mas reúne responsabilidades demais.

Também não basta juntar classes que mudaram juntas uma vez. Proximidade física pode diminuir dependências entre arquivos ou diretórios na medição escolhida, mas reduzir coesão desses agrupamentos se misturar conceitos diferentes. Analise responsabilidades e padrões reais de mudança.

Pratique antes de continuar

Q38. Dê um exemplo de dependência entre classes sem referência direta e uma forma de torná-la mais controlável.

Q39. Um programador move para o mesmo arquivo todas as classes alteradas na mesma tarefa. Isso garante melhor projeto? Analise coesão, dependência e granularidade.

Q40. Explique classitis e refatore conceitualmente a sequência “`new Circle(); setRadius(5); getArea()`” para reduzir uma dependência temporal evitável.

9.2 CBO e LCOM1: método para não errar

CBO (Coupling Between Objects) conta classes distintas às quais uma classe está acoplada segundo a convenção utilizada. No questionário 4, herança, tipos de atributos e de parâmetros entram na contagem. Não conte a mesma classe novamente porque aparece em vários lugares.

```
class A extends B {
    C f1, f2, f3;
    void m1(D p) { /* usa f1 e f2 */ }
    void m2(E p) { /* usa f2 e f3 */ }
    void m3(F p) { /* usa f3 */ }
}
```

As classes são B, C, D, E e F. **CBO = 5**. Conte dependências distintas, não atributos ou chamadas. Em exercícios diferentes, explicita tratamento de tipos básicos, bibliotecas, construtores e herança se o enunciado não definir.

LCOM1: pares de métodos e atributos comuns

LCOM1 = max(P - Q, 0). P é o número de pares sem atributo compartilhado; Q é o número de pares com pelo menos um atributo compartilhado. Com n métodos, existem $n(n - 1)/2$ pares não ordenados: m1/m2 e m2/m1 são o mesmo par.

Par do exemplo	Atributo comum	Classificação
m1/m2	f2	Q
m1/m3	Nenhum	P
m2/m3	f3	Q

P = 1 e Q = 2. Portanto, **LCOM1 = max(1 - 2, 0) = 0**. Compartilhar três atributos continua contando como um par Q; não como três. Não conte um método comparado consigo mesmo.

Se três métodos usam o mesmo atributo x, P = 0 e Q = 3, logo LCOM1 = 0. Se cada um usa exclusivamente x, y ou z, P = 3 e Q = 0, logo LCOM1 = 3. O segundo caso sugere responsabilidades menos relacionadas.

INTERPRETAÇÃO: LCOM1 maior indica mais falta de coesão segundo essa métrica. Zero é favorável, mas não prova responsabilidade única ou design perfeito. CBO também não mede sozinho a qualidade de cada dependência.

Pratique antes de continuar

Q41. Uma classe herda de Base, possui três atributos do tipo Repo e métodos com parâmetros Cliente e Produto. Conte CBO incluindo herança e esses tipos, sem outras dependências.

Q42. Quatro métodos usam: m1={a,b}; m2={b}; m3={c}; m4={c,d}. Liste os seis pares e calcule LCOM1.

Q43. Dois métodos compartilham cinco atributos. Quanto esse par acrescenta a Q? Um LCOM1 igual a zero permite dispensar análise das responsabilidades?

10. Princípios: SRP, ISP e DIP

Propriedades descrevem características desejadas; princípios orientam decisões para alcançá-las. São recomendações a aplicar com contexto, não regras para criar abstrações sem necessidade.

Responsabilidade Única (SRP)

Uma classe deve ter uma responsabilidade coesa, um motivo principal para mudar. Misturar calcular índice, imprimir relatório e acessar banco reúne razões independentes de alteração. Separar funções sem separar efetivamente responsabilidades pode apenas espalhar o problema.

No exemplo bancário do questionário, onclick lê campos, busca contas, executa retirada e depósito e controla transação. Uma organização melhor deixa a interface coletar dados e exibir resultado; um serviço coordena a transferência e sua transação; repositórios encapsulam acesso; objetos de domínio preservam regras.

A transação precisa abranger retirada e depósito. Separar o código não deve destruir a atomicidade: se uma parte falhar, a transferência não pode ficar pela metade.

Segregação de Interfaces (ISP)

Clientes não devem depender de operações de que não precisam. Uma interface gigantesca com imprimir, digitalizar e enviar fax pode obrigar uma impressora simples a implementar funções sem sentido. Interfaces específicas permitem que cada cliente dependa do serviço necessário.

O objetivo não é “uma interface por método” indiscriminadamente. A divisão deve acompanhar grupos coesos de operações e necessidades dos clientes.

Inversão de Dependências (DIP)

Módulos de alto nível e detalhes devem depender de abstrações adequadas. Nos slides, a orientação aparece como **prefira interfaces a classes**. Um controle remoto que depende de TVGenerica pode operar com implementações diferentes sem conhecer cada marca.

```
interface Pagamento { void pagar(Valor valor); }
class Checkout {
    private Pagamento pagamento;
    Checkout(Pagamento p) { pagamento = p; }
    void concluir(Valor v) { pagamento.pagar(v); }
}
```

O checkout depende do contrato; um adaptador integra cada fornecedor. Receber a dependência por construtor é uma técnica útil, mas DIP não se resume a usar um contêiner de injeção. Um script pequeno, estável e sem necessidade de substituição pode não justificar camadas adicionais.

Pratique antes de continuar

Q44. Um clique de tela lê campos, consulta banco, retira dinheiro, deposita e executa commit. Identifique o princípio e proponha uma separação que preserve a transação.

Q45. Uma interface de leitura exige que um cliente implemente também exclusão e exportação, que ele não usa. Qual princípio aplicar e como?

Q46. Como preparar um sistema para trocar o fornecedor de pagamento? Explique por que não basta renomear a classe concreta para “PagamentoGenerico”.

10.1 Aberto/Fechado, composição e Demeter

Aberto/Fechado (OCP)

Um módulo deve permitir extensões relevantes sem exigir que clientes editem sua lógica interna a cada variação. Parâmetros, funções, estratégias e polimorfismo podem criar pontos de extensão. Não significa que o código nunca possa ser corrigido ou refatorado.

Um algoritmo de ordenação que recebe um comparador permite ordenar nomes por tamanho sem reescrever o algoritmo. Um cálculo de frete que recebe uma estratégia permite acrescentar uma modalidade implementando esse contrato. A abstração deve corresponder a uma variação útil, sem tentar prever todas as mudanças imagináveis.

Prefira composição a herança

Herança representa uma relação “é um” compatível com os contratos da base. Composição representa “possui/usa”: um painel possui um conta-giros. Quando a intenção é apenas reutilizar código, composição e delegação frequentemente evitam uma hierarquia rígida.

É possível ter polimorfismo sem herança de implementação, usando interfaces ou tipagem estrutural. Por isso, no exercício de escolher apenas dois mecanismos entre encapsulamento, polimorfismo e herança, uma defesa consistente é preservar os dois primeiros e substituir parte do reuso por composição.

Lei de Demeter: menor conhecimento

Evite obrigar um cliente a atravessar objetos de passagem. Em “biblioteca.getAcervo().getArea().getLivros().buscar()”, ele conhece a organização interna da biblioteca. Uma operação como biblioteca.consultarExemplares(area, livro) pode concentrar esse conhecimento onde pertence.

```
// Conhecimento externo da estrutura interna:  
Cliente c = conta.getCliente();  
String email = c.getEmailAddress();
```

```
// Uma alternativa para o cliente dessa API:  
conta.enviarEmailAoCliente(mensagem);
```

O método exposto pode delegar a um serviço de comunicação. A meta é reduzir o conhecimento externo, não obrigar um objeto de domínio a implementar toda a infraestrutura de envio.

Trocar uma cadeia por variáveis temporárias não remove a dependência. Por outro lado, uma API fluente, como map().filter().reduce(), não é automaticamente ruim: não está necessariamente expondo uma cadeia de objetos internos. O próprio material recomenda bom senso.

Pratique antes de continuar

Q47. Um método de ordenação recebe um comparador. Qual princípio isso favorece? Explique por que OCP não proíbe corrigir defeitos.

Q48. Reorganize conceitualmente “empresa.getDepartamento().getGerente().getEmail()” quando o cliente só quer enviar um aviso ao gerente. Nomeie o princípio.

Q49. Você precisa abrir mão de herança, encapsulamento ou polimorfismo. Defenda uma escolha e explique como compensaria a perda. Não basta dizer “herança é ruim”.

10.2 Liskov e contratos de comportamento

O **Princípio da Substituição de Liskov (LSP)** exige que um objeto de subtipo possa ser usado onde o tipo base é esperado sem quebrar as expectativas válidas do cliente. Herdar sintaxe e métodos não garante substituição correta.

Pré-condição e pós-condição

Pré-condição é uma exigência para chamar uma operação. **Pós-condição** é uma garantia após sua execução válida. A subclasse não deve fortalecer a pré-condição nem enfraquecer a pós-condição. Deve também preservar invariantes relevantes do contrato.

REGRA PARA LEMBRAR: a subclasse pode exigir menos e garantir mais. Não pode exigir mais nem garantir menos que a base.

Exemplo resolvido do questionário

Classe A aceita $x > 0$ e garante resultado > 0 . Classe B, que herda de A, aceita $x > 10$ e garante resultado > -50 .

Pré-condição: B rejeita entradas positivas menores ou iguais a 10 que A aceitava. Um cliente que passou $x = 5$ cumpriu o contrato de A, mas falha ao receber B. **Pós-condição:** B permite resultados negativos ou zero, contrariando a garantia positiva de A. São duas violações independentes.

Uma redefinição que aceite $x \geq 0$ e mantenha resultado > 0 não impõe exigência maior às chamadas válidas da base. Uma garantia resultado > 10 é mais forte que resultado > 0 . O raciocínio depende dos conjuntos de entradas aceitas e saídas permitidas, não de uma palavra isolada como “maior”.

Outro exemplo

Se um controle remoto promete operar a 10 metros, um subtipo que só opera a 5 metros não mantém a promessa. Um que opera a 20 metros pode atender à expectativa nesse aspecto. Isso não prova conformidade de todos os outros comportamentos: a comparação precisa considerar o contrato completo pertinente.

Diferencie LSP de outros princípios

Um método que mistura interface e banco aponta para SRP. Um cliente que conhece objetos internos aponta para Demeter. Uma subclasse que rejeita operações válidas da base aponta para LSP. Um exemplo pode envolver mais de um princípio, mas priorize a evidência central do enunciado.

Pratique antes de continuar

Q50. A base aceita $x \geq 0$ e garante $y \geq 10$. A subclasse aceita $x \geq 5$ e garante $y \geq 0$. Analise separadamente pré e pós-condição e use um contraexemplo.

Q51. A base aceita $x > 10$ e garante $y > 0$. A subclasse aceita $x > 0$ e garante $y > 20$. Há violação nesses dois aspectos? Justifique por inclusão de valores.

Q52. Uma subclasse herda uma operação da base, mas passa a rejeitá-la sempre com “operação não suportada”. Quando isso caracteriza violação de Liskov?

11. Integração: da necessidade ao código

Imagine um produto para reservar salas. Há incerteza sobre a demanda e sobre o fluxo; depois, surgem problemas de qualidade do código. Cada técnica resolve uma parte, e confundi-las leva a decisões inadequadas.

Uma sequência possível

Primeiro, esclareça a necessidade com stakeholders. Se o uso real ainda é incerto, organize uma experiência mínima de reservas e observe demanda. Para o fluxo, crie um protótipo que mostre encontrar sala, verificar capacidade e confirmar. Teste com estudantes usando tarefas sem instruções de interface.

Transforme o aprendizado em histórias e critérios. O PO ordena o backlog; os Developers selecionam trabalho e planejam o Sprint. Integração contínua, testes e revisão sustentam qualidade. Limites de WIP ajudam a evitar muitas reservas “quase prontas” aguardando teste.

No código, a interface solicita a reserva; um serviço coordena a operação; o domínio protege regras; o acesso a dados fica encapsulado. A Review discute o resultado do produto; a Retrospectiva examina, por exemplo, a fila recorrente em validação.

Complementos que podem aparecer na prova

Frequência de releases. A questão 8 do questionário 2 apresenta a ordem didática: Waterfall, desenvolvimento evolucionário, ágil, Facebook no estudo citado e Continuous Deployment. A resposta registrada associa, respectivamente, entrega ao final, poucos meses, poucas semanas, entregas diárias/semanais e intervalos inferiores a uma hora. São referências históricas daquele exercício, não regras universais ou descrição atual das empresas.

Integração contínua integra e verifica mudanças frequentemente. **Entrega contínua** mantém o produto em condição de ser disponibilizado. **Implantação contínua** automatiza a disponibilização das mudanças aprovadas pelo fluxo. Um Sprint pode conter várias liberações; não é obrigatório esperar sua Review.

Pratique antes de continuar

Q53. Um produto novo tem dúvida de demanda e de navegação. Como combinar MVP, histórias e protótipo sem tratar tudo como o mesmo artefato?

Q54. O PO priorizou “editar texto” antes de “abrir arquivo”. Dê uma alternativa técnica para testar valor e uma situação que exigiria renegociar a ordem.

Q55. Documentação de API fica desatualizada repetidamente. Qual mudança de prática e qual evento Scrum podem ajudar?

Q56. Organize as cinco abordagens de frequência de releases citadas acima. Por que não tratar os intervalos como regras universais?

12. Revisão de véspera: reconhecer e justificar

Se o enunciado mostrar...	Pense primeiro em...
Etapas cheias e trabalho esperando	Gargalo, pull e WIP
Pontos de times diferentes	Escala relativa; não comparar produtividade
Fluxo incerto antes da implementação	Prototipação e aprendizado barato
Demanda de mercado desconhecida	Hipótese e MVP
Usuário conclui só após dica	Facilitação enviesada; sucesso com ajuda
Classe com responsabilidades diferentes	Coesão e SRP
Navegação por objetos internos	Demeter e ocultamento
Subclasse exige mais ou garante menos	Liskov
Troca frequente de fornecedor	Abstração e DIP
Variação por comparador/estratégia	Aberto/Fechado

Fórmulas essenciais

WIP = Throughput × Lead Time, usando médias, regime e unidades coerentes. **LCOM1 = $\max(P - Q, 0)$** . **Pares de métodos = $n(n - 1)/2$** . **Conversão = convertidos/expostos**. **Ganho relativo = $(\text{taxa B} - \text{taxa A})/\text{taxa A}$** . Diferença entre percentuais se expressa em pontos percentuais.

Erros que tiram qualidade de uma resposta

Usar uma palavra sem justificar; tratar exemplos numéricos dos slides como leis; afirmar que ágil dispensa disciplina; confundir Sprint Review com Retrospectiva; tratar CBO como contagem de atributos; contar pares repetidos em LCOM; exigir protótipos completos; supor que sucesso com ajuda demonstra clareza; mudar apenas nomes e arquivos sem resolver dependências.

Antes de entregar uma discursiva, pergunte: respondi todos os verbos do enunciado? Dei o exemplo pedido? Expliquei consequência? A melhoria resolve o problema sem criar outro? Se houve conta, interpretei o resultado?

Última rodada antes do simulado

Q57. Explique em até quatro frases a diferença entre Scrum e XP e como ambos podem ser usados com Kanban.

Q58. Escreva uma resposta completa para: “Todos os métodos da classe usam o atributo x, então a classe tem design perfeito”. Use métrica e interpretação.

Q59. “Como o protótipo passou no teste com cinco pessoas, segurança e desempenho estão validados”. Avalie e indique o que realmente se pode concluir.

Q60. Uma equipe corta testes para cumprir todos os itens selecionados no Sprint. Analise qualidade, objetivo, negociação de escopo e DoD.

PRÓXIMO PASSO: faça o simulado sem consulta em 45 a 60 minutos. As respostas estão somente a partir da página 31. Para uma avaliação útil, registre o raciocínio mesmo quando não tiver certeza.

13. Simulado final - parte 1

Tempo sugerido: 45 a 60 minutos para as 12 questões. Total: 24 pontos, dois por questão. Use papel separado. O gabarito traz critérios de pontuação e aceita respostas equivalentes bem justificadas.

Q61. Escolha do processo

Uma empresa quer criar um serviço digital, mas ainda não conhece todas as necessidades dos clientes. O gerente propõe seis meses de especificação, seis de implementação e uma apresentação final. Compare essa estratégia com ciclos curtos de entrega e feedback. Explique dois riscos e uma vantagem contextual do planejamento antecipado.

Q62. Scrum na prática

Durante um Sprint, o patrocinador exige novas funcionalidades e ordena que o Scrum Master distribua tarefas e prorogue o Sprint. O objetivo original continua válido. Como o time deve lidar com a demanda? Mencione PO, Developers, Scrum Master, escopo e duração.

Q63. Fluxo e cálculo

Um processo estável conclui 4 itens por dia e mantém 20 itens de WIP médio. Calcule seu Lead Time. Há uma fila na validação; um gestor sugere iniciar mais itens na implementação para “aumentar a produtividade”. Avalie a proposta e recomende uma ação.

Q64. História e critérios

Escreva uma história para um aluno consultar o resultado de uma solicitação de revisão de nota. Apresente dois critérios de aceitação e um critério de DoD que não repita os critérios funcionais.

Q65. MVP e incremento

Uma startup entrega autenticação e perfil na primeira Sprint e chama isso de MVP. Qual informação falta para sustentar essa classificação? Proponha um experimento mínimo para testar se estudantes contratariam monitoria paga.

Q66. Experimento A/B

Em A, 200 de 4.000 usuários expostos compram. Em B, 240 de 4.000 compram. Calcule as duas taxas, a diferença absoluta e o ganho relativo. Explique por que a análise não deve terminar apenas com “B teve mais compradores”.

13. Simulado final - parte 2

Q67. Escolha e teste de protótipo

Uma equipe quer saber se idosos encontram a função de reagendar atendimento. Ela testa um mockup estático apenas com os próprios desenvolvedores, depois de explicar todas as telas. Proponha três mudanças no teste e justifique cada uma.

Q68. Interpretação de evidências

Cinco pessoas tentam cancelar uma reserva. Duas concluem sem ajuda; duas concluem após dicas; uma abandona. A equipe anuncia “80% de sucesso; pronto para implementar”. Analise a conclusão e indique uma decisão e dois dados adicionais relevantes.

Q69. Métricas de código

Considere somente as dependências e os métodos indicados; inclua herança no CBO e desconsidere construtores implícitos.

```
class Relatorio extends Base {  
    Repo a;  
    Repo b;  
    Cache c;  
    void m1(Cliente p) { /* usa a */ }  
    void m2(Cliente p) { /* usa a e b */ }  
    void m3(Filtro p) { /* usa c */ }  
}
```

Calcule CBO e LCOM1. Liste classes distintas e pares de métodos. Interprete os resultados sem concluir automaticamente que o projeto é bom ou ruim.

Q70. Responsabilidades e dependências

Um método de interface lê campos, consulta tabelas, calcula desconto, envia e-mail e retorna mensagem visual. Identifique o problema principal e proponha uma divisão. Explique por que mover cada linha para uma classe separada pode criar outro problema.

Q71. Substituição de Liskov

A base aceita quantidade ≥ 1 e garante prazo ≤ 5 dias. A subclasse aceita quantidade ≥ 10 e garante prazo ≤ 8 dias. Analise separadamente as duas condições e dê um exemplo de cliente prejudicado.

Q72. Projeto para evolução

O código cliente executa “loja.getPedido().getCliente().getEmail()” para notificar uma compra, e o envio está preso a um fornecedor concreto que será trocado. Proponha duas melhorias, nomeie os princípios correspondentes e explique uma limitação de aplicar abstrações indiscriminadamente.

Fontes, escopo e pontos de atenção

Esta apostila foi elaborada a partir dos arquivos do ZIP informado. As explicações integram os temas e acrescentam exemplos didáticos; as 72 questões são adaptações ou exercícios autorais alinhados ao nível observado. As respostas dos arquivos TXT não são tratadas como gabarito oficial quando não há identificação de correção.

Mapa das fontes

Arquivo ou grupo	Uso nesta apostila
Aula 1 - Introdução da disciplina	Fundamentos, testes, manutenção, sistemas ABC
Aula 2 - Introdução a Processos de Desenvolvimento	Cascata, agilidade, UP, Espiral e V
Aula 3 - Processos de Desenvolvimento (XP)	Valores, práticas, pares e contratos
Aula 4 - Processos de Desenvolvimento (Kanban)	Pull, quadros, WIP e gargalos
Aula 5 - Processos de Desenvolvimento (Scrum)	Organização, eventos e estimativas
Aula 6 - Requisitos (Histórias de Usuário)	3 Cs, INVEST, aceitação e DoD
Aula 7 - Requisitos (Casos de Uso, MVP, Testes A_B)	Fluxos, experimentos e conversão
Aulas 8 e 9 - Prototipação e Testes	Artefatos, participantes e interpretação
Aula 10 - Propriedades de Projeto	Coesão, acoplamento e ocultamento
Aula 11 - Princípios de Projeto	SRP, ISP, DIP, OCP, Demeter e LSP
Guia do Scrum 2020, versão portuguesa europeia	Referência normativa de Scrum nesta revisão
Questionário 1 e Questionários 2, 3 e 4	Calibração de dificuldade e situações de treino

Ressalvas para estudar sem decorar inconsistências

Scrum: Guia 2020, páginas impressas 5 a 12 (6 a 13 do PDF), esclarece responsabilidades, eventos e artefatos. As contagens reduzidas e a faixa de tamanho do time em partes da aula 5 não substituem essa definição. Sprint de 15 dias, roteiro de três perguntas e uso de Story Points são exemplos/práticas, não exigências universais.

Prototipação: Q2 do questionário 3 tem ambiguidade sobre a conclusão de P4. Preserve a evidência de dificuldade e não invente um desfecho. Limiares de 80% e amostras de cinco participantes nos exemplos não são regras gerais de validação.

Métricas: a convenção de CBO inclui herança nos exercícios trabalhados. LCOM1 zero é um indicador limitado. A Lei de Little usa médias e fronteiras coerentes; o resultado não define automaticamente limites operacionais.

A/B e releases: o tamanho exato de amostra e os intervalos históricos de disponibilização são apresentados conforme as respostas registradas no questionário 2. O artigo externo citado naquele exercício não veio no ZIP; a descrição não é uma verificação independente nem uma afirmação sobre práticas atuais de empresas.

A seguir: gabarito comentado. Faça uma tentativa própria antes de consultar.

Gabarito comentado - Q01 a Q08

Q01. Renovar empréstimos é funcional; atender 2.000 usuários simultâneos é não funcional, ligado a capacidade; “fácil de usar” é uma intenção não funcional de usabilidade, mas vaga. Uma formulação possível: “No teste definido, ao menos 8 de 10 usuários representativos devem renovar um empréstimo sem ajuda em até 2 minutos”. O limiar é um exemplo a negociar, não uma norma. O essencial é indicar tarefa, público, condição e medida verificável.

Q02. Script pessoal descartável: C, pelo baixo impacto. Matrícula: B, pois sustenta atividade importante da universidade. Controlador de freios: A, porque uma falha pode comprometer vidas. A justificativa deve usar criticidade e consequência, não “é A porque tem muito código”.

Q03. Não. Os 300 testes cobrem condições escolhidas; outras entradas, sequências, estados ou ambientes podem revelar falhas. Unidade verifica um componente pequeno de forma isolada; integração verifica colaboração, como serviço e banco. Passar nos testes aumenta confiança dentro da cobertura efetiva, sem prova geral de ausência de bugs.

Q04. A extração que preserva comportamento é refatoração. A alteração da regra muda comportamento e não é, por esse fato, refatoração. Novas pessoas exigem treinamento, coordenação e redistribuição; tarefas próximas da entrega podem não ser divisíveis. Cite essas causas, evitando apenas repetir a Lei de Brooks.

Q05. Engenharia tradicional oferecia uma referência estabelecida de planejamento, documentação e execução sequencial. Com necessidades incertas, longos períodos sem avaliar software podem consolidar requisitos errados. O erro descoberto na entrega afeta projeto, implementação e testes, elevando o retrabalho. Planejamento não é inútil; o problema é adiar aprendizado relevante.

Q06. Iteração significa repetir ciclos; agilidade também envolve colaboração, adaptação e entrega frequente de valor. Documentação útil continua importante e deve ser proporcional ao contexto. UP e Espiral permitem ciclos e evolução; cascata é predominantemente sequencial. Na comparação dos slides, aqueles modelos preservam mais planejamento/documentação e ciclos mais longos, portanto iteração isolada não os torna ágeis.

Q07. Integração contínua detecta incompatibilidades cedo; testes automatizados detectam regressões de forma repetível; refatoração reduz fragilidade interna. Também cabem TDD, para orientar implementação por comportamentos, e pares, para revisão e compartilhamento. Ganhe qualidade explicando qual risco cada prática enfrenta; listar três nomes não basta.

Q08. Escopo fechado define entregas negociadas; aberto permite ajustar o conteúdo ao longo de períodos de trabalho e capacidade acordada. Se prazo, custo e escopo não podem mudar, surpresas podem ser absorvidas pela redução de testes ou cuidados, prejudicando qualidade. Escopo aberto exige acompanhamento do cliente. Pares podem reduzir erros e concentrar conhecimento compartilhado, mas exigem coordenação e dedicação simultânea de duas pessoas.

AUTOAVALIAÇÃO: você consegue explicar quando uma prática ajuda e qual problema ela enfrenta? Se suas respostas forem apenas definições, refaça Q05 e Q07 com um exemplo concreto.

Gabarito comentado - Q09 a Q16

Q09. O PO ordena valor e esclarece necessidades; não define unilateralmente como os Developers trabalham nem obriga uma seleção incompatível com a capacidade. Developers selecionam e planejam em discussão com ele. O SM apoia eficácia e autogerenciamento. A demanda do diretor deve ser discutida e priorizada, com transparência sobre capacidade, sem transformar o PO em chefe técnico.

Q10. PO esclarece a história e critérios; Developers investigam o ambiente e adaptam o plano; SM promove a remoção do impedimento, mobilizando quem pode ajudar. Transparência torna bloqueio e dúvida visíveis; inspeção avalia impacto; adaptação muda o plano ou resolve o obstáculo. Não é necessário que o SM conserte pessoalmente o banco.

Q11. Review inspeciona o produto e adapta próximos passos, por exemplo priorizar uma melhoria após feedback. Retrospectiva melhora a maneira de trabalhar, por exemplo mudar revisão de código. A Daily não exige as três perguntas tradicionais: os Developers escolhem uma estrutura útil para inspecionar progresso e ajustar o plano rumo ao Sprint Goal.

Q12. Apenas o PO tem autoridade para cancelar se o Sprint Goal se tornar obsoleto. Se uma tarefa atrasou, o time deve avaliar e adaptar o plano, podendo renegociar escopo com o PO sem ameaçar o objetivo ou reduzir qualidade. Não se cancela nem se prorroga automaticamente um Sprint por atraso isolado.

Q13. Não integra o incremento se descumpre a DoD. A revisão obrigatória é parte do padrão de qualidade compartilhado. Um critério de aceitação não atendido significa que um resultado específico da história ainda está incorreto ou incompleto. São níveis diferentes de avaliação; “funcionou na minha máquina” não substitui nenhum deles.

Q14. Não há base para comparação universal de produtividade: escalas e contextos podem ser diferentes. Estimativas ajudam a discutir entendimento, dividir itens e planejar trabalho compatível com a capacidade histórica. O Sprint Backlog reúne o Sprint Goal, os itens selecionados e o plano para entregá-los, não apenas uma lista de tarefas.

Q15. Priorize liberar a etapa mais à frente que está bloqueando o fluxo: colaborar para concluir validação com qualidade. Isso permite puxar itens implementados e liberar etapas anteriores. Aumentar WIP pode elevar filas e tempo de atravessamento sem mudar a capacidade do gargalo. Uma alteração de limite precisa de diagnóstico e acompanhamento.

Q16. Sim: $3 + 2 = 5$ itens dentro de uma etapa de limite 4, pela convenção do quadro intermediário dos slides. Itens aguardando a próxima etapa continuam ocupando WIP. Cartões podem retornar por defeitos ou retrabalho conforme políticas explícitas. A equipe deve manter o trabalho visível, analisar a causa e respeitar os critérios do fluxo.

ERRO FREQUENTE: confundir “não programar uma tarefa nova agora” com ociosidade. Ajudar a concluir, revisar e remover bloqueios pode gerar mais valor que aumentar o estoque de trabalho iniciado.

Gabarito comentado - Q17 a Q24

Q17. $WIP = 3 \text{ tarefas/dia} \times 4 \text{ dias} = 12 \text{ tarefas}$. Com WIP 18 e vazão 3, $Lead\ Time = 18/3 = 6 \text{ dias}$. A interpretação é mais tempo médio dentro do fluxo sem aumento de vazão. Declare a hipótese de regime estável.

Q18. $Lead\ Time = 15 \text{ itens} / 10 \text{ itens por semana} = 1,5 \text{ semana}$. É uma média observada. O limite de WIP depende de variabilidade, capacidade, políticas e objetivo do fluxo. Não se deve transformar diretamente uma relação entre médias em configuração ótima.

Q19. Exemplos: “Como estudante, quero renovar um empréstimo para continuar usando o livro”; “Como estudante, quero reservar um título indisponível para entrar na fila de retirada”; “Como bibliotecário, quero registrar a devolução para atualizar disponibilidade e situação do empréstimo”. Aceitam-se histórias equivalentes com papel real, necessidade e benefício coerentes.

Q20. “Como professor, quero criar uma prova para avaliar conteúdos”; “Como aluno, quero realizar a prova online para registrar minhas respostas”; “Como professor, quero consultar resultados para identificar dificuldades da turma”. Tarefa técnica: criar tabela de respostas. Spike: investigar, por um período definido, alternativas de correção automática e apresentar uma recomendação. O spike reduz incerteza; não promete a funcionalidade pronta.

Q21. “Moderno” é vago e “banco novo” impõe uma solução técnica sem necessidade explicitada. Uma hipótese de reescrita: “Como estudante, quero consultar minhas solicitações em aberto para acompanhar o que ainda depende de resposta”. Deve-se validar se essa necessidade é realmente a pretendida; não se pode deduzir o requisito real apenas da frase ruim.

Q22. Aceitação: o criador consegue fechar o formulário; novas respostas são recusadas depois do fechamento; respostas anteriores permanecem disponíveis. DoD: testes relevantes passando e revisão de código concluída; atualização de documentação quando necessária também cabe. “Ter botão de fechar” sozinho não comprova o bloqueio de novas respostas.

Q23. Pesquisa sem resultados: informar e permitir nova busca. Carrinho com item sem estoque: informar e impedir compra inválida ou permitir ajustar, conforme a regra definida. Endereço inválido: apresentar motivo e permitir correção. Pagamento recusado: informar, preservar carrinho e oferecer outra forma sem confirmar compra. Cada extensão deve indicar claramente sua etapa; outras alternativas coerentes são válidas.

Q24. a) Histórias para pequenas melhorias e prioridade frequente. b) Casos de uso para esclarecer fluxo, regras e exceções. c) MVP para testar hipótese de demanda com investimento mínimo. Podem ser combinados: o MVP contém necessidades expressas em histórias e fluxos detalhados quando necessário; protótipos podem avaliar sua interação. Técnica adequada depende da pergunta, não de uma exclusão rígida.

Gabarito comentado - Q25 a Q32

Q25. Hipótese: estudantes usarão caronas combinadas entre campus e bairros. Operação mínima: formulário para pedidos e ofertas, com combinação manual por mensagens e regras claras. Métrica: proporção de pedidos que viram viagens efetivamente realizadas, em período e público definidos. Decisão: continuar, ajustar horários/público ou interromper conforme evidências. Número de curtidas não equivale a caronas realizadas.

Q26. Não necessariamente. Cadastro e login podem ser apenas infraestrutura de um incremento. Falta explicitar a hipótese e a evidência produzida com usuários reais. Pivô: trocar de público generalista para universitários após observar demanda nesse grupo. Melhoria pequena: esclarecer um rótulo confuso. O pivô muda uma direção relevante; não é qualquer ajuste.

Q27. $A = 100/2.000 = 5\%$; $B = 120/2.000 = 6\%$. Ganho absoluto = **1 ponto percentual**; relativo = $1/5 = 20\%$. A superioridade estatística exige análise apropriada, desenho válido e consideração da incerteza; a taxa maior sozinha não basta. Evite chamar 1 ponto percentual de ganho relativo de 1%.

Q28. Eventos repetidos não devem virar pessoas convertidas em uma taxa definida por usuário. Conte usuários únicos que converteram ou declare outra métrica, como cliques por pessoa. Encerrar no primeiro dia favorável pode inflar conclusões falsas no teste convencional. Planeje amostra e regra de análise; mantenha atribuição consistente e analise a diferença com a incerteza correspondente.

Q29. a) Sketch para explorar rápido; wireframe também pode ser adequado se o foco é organização estrutural. b) Mockup para aparência. c) Protótipo interativo para observar a execução do fluxo. A justificativa deve relacionar o artefato à pergunta, e não apenas dizer “é mais bonito” ou “mais completo”.

Q30. Construir toda a infraestrutura antes de validar pode elevar investimento em um fluxo errado e tornar mudança cara. Um protótipo focado reduz risco ao testar a sequência principal com menos custo. O escopo deve incluir as ações necessárias ao aprendizado, simulando detalhes fora da pergunta. Não é proibido implementar primeiro quando a dúvida é técnica; a escolha deve acompanhar a incerteza real.

Q31. Objetivo: verificar cancelamento sem ajuda. Cenário: “Sua reunião de amanhã foi desmarcada e você não precisa mais da sala reservada”. Tarefa: “Cancele a reserva”. Evidências: conclusão independente, cliques errados, hesitações, tempo, pedidos de ajuda e entendimento da confirmação. Não mencionar nomes de menus preserva a oportunidade de observar descoberta.

Q32. Explicar telas ensina o caminho; indicar calendário dirige a ação; registrar sucesso sem distinguir ajuda distorce o resultado. Apresente apenas contexto e tarefa, observe a tentativa, faça perguntas neutras e registre ajuda. Também não generalize uma sessão isolada para todo o público. A correção central é separar demonstração de teste.

Gabarito comentado - Q33 a Q40

Q33. A taxa de conclusão total pode ser 80%, mas não demonstra autonomia: apenas uma das cinco pessoas concluiu sem ajuda, ou **20%**. Registre separadamente conclusão com e sem ajuda, obstáculos, cliques, tempo e confiança. Corrija dificuldades recorrentes e teste novamente. Não basta simplesmente aumentar participantes mantendo o mesmo problema de facilitação.

Q34. Priorize enfermeiros que representam rotina, linguagem e contexto de uso. Desenvolvedores podem detectar questões técnicas e administrativos, problemas gerais, mas não substituem o público-alvo nessa avaliação. O tamanho do grupo, isoladamente, não compensa sua inadequação ao objetivo.

Q35. Pergunte o que a pessoa queria fazer, o que esperava encontrar e em qual momento sentiu falta. Observe se existe problema de visibilidade, rótulo ou fluxo antes de adicionar outro botão. Alta fidelidade não corrige amostra inadequada, tarefa enviesada ou facilitação que ensina o caminho; pode até tornar alterações mais caras.

Q36. Prejudica ocultamento de informação e proteção de regras, pois clientes manipulam a estrutura diretamente. Exponha operações do domínio, como registrarEntrada, registrarSaida e consultarVeiculo, mantendo o armazenamento privado. Trocar mapa por banco passa a exigir mudanças concentradas na implementação da classe, em vez de em todos os clientes.

Q37. Métodos diferentes podem atender à mesma responsabilidade: push, pop e size operam sobre uma pilha. Alta coesão é relação de propósito, não contagem mínima de métodos. Falta de integridade conceitual: misturar sem motivo snake_case e camelCase, ou usar "sair" com comportamento diferente em telas equivalentes.

Q38. A grava um CSV e B lê a mesma estrutura. A mudança de ordem ou significado de campo quebra B mesmo sem chamada direta. Um serviço com contrato estável, esquema explícito e evolução controlada torna a dependência mais visível e administrável. Colocar as classes próximas não elimina a necessidade de contrato.

Q39. Não garante. Pode aproximar código relacionado e reduzir dependências entre arquivos na métrica escolhida, mas pode juntar responsabilidades incompatíveis e criar arquivos enormes. O acoplamento entre as classes continua existindo. É necessário examinar motivos de mudança e o nível de modularização avaliado, não usar coincidência de uma alteração como regra absoluta.

Q40. Classitis é fragmentação excessiva: muitas classes aparentemente simples aumentam navegação, interfaces e relações necessárias para entender uma funcionalidade. Para o círculo, usar new Circle(5) pode garantir um estado válido desde a construção, eliminando a obrigação externa de chamar setRadius antes de getArea. A solução deve validar o raio conforme o contrato.

AUTOAVALIAÇÃO: diferencie um defeito observado, uma interpretação possível e a solução proposta. Essa separação melhora tanto respostas sobre testes de usuários quanto sobre design de código.

Gabarito comentado - Q41 a Q46

Q41. As classes distintas são **Base, Repo, Cliente e Produto**. Logo, **CBO = 4**, incluindo herança conforme solicitado. Repo aparece três vezes, mas conta uma. Não conte a própria classe ou tipos não indicados na convenção do exercício.

Q42. Há seis pares: m1/m2 compartilha b (Q); m1/m3 não compartilha (P); m1/m4 não compartilha (P); m2/m3 não compartilha (P); m2/m4 não compartilha (P); m3/m4 compartilha c (Q). Assim, **P = 4; Q = 2; LCOM1 = $\max(4 - 2, 0) = 2$** . Há dois grupos pouco conectados segundo o uso de atributos, um indício para analisar separação de responsabilidades.

Q43. Acrescenta **1 a Q**, pois se contam pares de métodos, não atributos comuns. Zero não dispensa análise: métodos podem acessar um atributo compartilhado e ainda reunir responsabilidades conceitualmente distintas. A métrica é um sinal estrutural limitado.

Q44. O problema central é violação de SRP/separação de responsabilidades. Interface coleta dados e mostra resultado; serviço de transferência coordena a operação e a transação; repositório busca e persiste; domínio aplica regras. Retirada e depósito devem estar no mesmo limite transacional pertinente para preservar atomicidade. Apenas extrair funções e deixar tudo conceitualmente misturado não resolve.

Q45. Aplique ISP. Separe um contrato de leitura dos contratos de exclusão e exportação quando atendem clientes distintos. O cliente depende apenas do que usa e não precisa criar operações vazias ou lançar exceções para cumprir uma interface inadequada. A divisão precisa continuar coesa; não deve gerar interfaces sem utilidade.

Q46. Defina uma abstração coerente de pagamento usada pelo sistema e implemente adaptadores para os fornecedores. Injete a implementação necessária na composição da aplicação. Isso favorece DIP e permite substituir detalhes preservando o contrato. Renomear uma classe concreta não muda suas dependências nem cria substituíbilidade. Avalie também diferenças reais dos fornecedores: um contrato simplificado demais pode não representar seus comportamentos.

Passo a passo para revisar cálculos

Em CBO, escreva primeiro um conjunto de nomes e elimine repetições. Em LCOM1, liste os pares antes de atribuir P e Q. Confira se $P + Q$ corresponde a $n(n - 1)/2$. Só então aplique $\max(P - Q, 0)$.

No exercício Q42, a conferência é $4 + 2 = 6$ e $4 \times 3/2 = 6$. Se a soma desse um valor diferente, haveria par faltando ou repetido. Esse teste simples evita boa parte dos erros de contagem.

Gabarito comentado - Q47 a Q52

Q47. Favorece Aberto/Fechado: o cliente acrescenta uma regra de comparação sem editar o algoritmo de ordenação. OCP orienta pontos de extensão para variações, não imutabilidade eterna do código. Corrigir um defeito ou melhorar implementação continua legítimo.

Q48. O cliente pode solicitar `empresa.notificarGerenteDoDepartamento(idDepartamento, mensagem)`, delegando a descoberta do gerente e a comunicação aos componentes responsáveis. Aplica-se Demeter, reduzindo conhecimento de objetos intermediários. A operação pode delegar internamente e não deve transformar Empresa numa classe que implementa toda a infraestrutura.

Q49. Uma defesa consistente é eliminar herança de implementação e manter encapsulamento e polimorfismo. Encapsulamento protege estado e decisões; polimorfismo permite diferentes comportamentos pelo mesmo contrato. Composição e delegação recuperam parte do reuso; interfaces ou tipagem estrutural permitem polimorfismo sem uma hierarquia de implementação. A escolha tem custos de delegação e não prova que herança seja sempre inadequada.

Q50. Pré-condição fortalecida: $x = 2$ era válido na base e é rejeitado na subclasse. Pós-condição enfraquecida: $y = 3$ é permitido pela subclasse, mas viola $y \geq 10$. São duas violações de LSP. Um cliente que fornece $x = 2$ ou depende de y ser pelo menos 10 pode deixar de funcionar.

Q51. Não há violação nesses aspectos: aceitar $x > 0$ inclui todas as entradas $x > 10$ exigidas pela base; garantir $y > 20$ implica $y > 0$. A subclasse exige menos e garante mais. Isso não basta para provar conformidade completa se houver outros comportamentos ou invariantes não apresentados.

Q52. Há violação se o contrato da base prometia a operação para aquelas condições válidas e a subclasse a rejeita. O cliente tinha uma expectativa legítima que deixa de ser atendida. Se o contrato já previa a restrição em questão, é necessário analisar essa previsão; não conclua apenas pela presença de uma exceção.

Contraexemplos são uma ferramenta de prova

Quando a subclasse parece incompatível, procure uma entrada aceita pela base e rejeitada por ela, ou uma saída permitida por ela e proibida pela base. Um único caso desse tipo pode demonstrar a quebra da expectativa correspondente.

Para uma garantia de prazo, cuidado: “prazo ≤ 3 ” é mais forte que “prazo ≤ 5 ”, pois promete entrega em menos tempo. O sentido de fortalecer ou enfraquecer depende do contrato, não de o número aumentar ou diminuir isoladamente.

Gabarito comentado - Q53 a Q60

Q53. Use MVP para observar demanda com uma experiência mínima real; histórias para expressar necessidades e orientar entregas; protótipo para investigar entendimento do fluxo. Os resultados se alimentam: o teste pode levar a dividir história e mudar critérios, enquanto o MVP pode revelar que o público ou a proposta precisa mudar. Cada artefato tem uma pergunta central.

Q54. Pode-se permitir editar um documento inicial ou conteúdo de exemplo, simulando a entrada enquanto se testa a edição. Se o valor da história depende essencialmente de abrir formatos reais ou de uma integração indispensável, explique a dependência e renegocie com o PO. Evite tanto desobedecer silenciosamente à prioridade quanto prometer algo inviável.

Q55. Discutir a causa na Retrospectiva e incorporar atualização de documentação relevante à DoD é uma ação coerente. Revisão de código e verificações do fluxo podem ajudar a garantir cumprimento. O objetivo é uma mudança de prática verificável, não apenas pedir “mais atenção” repetidamente.

Q56. Waterfall; evolucionário; ágil; Facebook conforme o estudo citado; Continuous Deployment. Os intervalos apresentados no arquivo são, respectivamente, final do projeto, poucos meses, poucas semanas, diário/semanal e menos de uma hora. São referências do exercício e variam por organização, época e processo; não descrevem necessariamente práticas atuais.

Q57. Scrum define estrutura de organização em Sprints, responsabilidades, eventos e artefatos. XP oferece práticas técnicas como testes, TDD, pares e integração contínua. Kanban pode ajudar a visualizar e limitar o trabalho em andamento dentro desse processo. O uso conjunto precisa preservar os objetivos de cada prática e responder aos problemas do time.

Q58. Todos compartilharem x pode resultar em LCOM1 = 0, pois todos os pares entram em Q. Isso é favorável à coesão medida pelo uso de atributos, mas não prova design perfeito. É preciso avaliar propósito dos métodos, qualidade do acoplamento, ocultamento de informação e clareza da interface.

Q59. Um teste com protótipo pode fornecer evidências de entendimento e navegação nas tarefas e perfis avaliados. Não verifica automaticamente segurança do código, desempenho real ou integrações. Esses aspectos exigem implementação e avaliações específicas. Mesmo usabilidade não está “provada para todos” por uma pequena rodada.

Q60. Cortar testes exigidos pela DoD para afirmar que todos os itens foram concluídos compromete a qualidade e a transparência. Os Developers devem adaptar o plano e discutir escopo com o PO, protegendo Sprint Goal e DoD. Trabalho que não atende à DoD não faz parte do incremento. Quantidade aparente de itens não substitui resultado utilizável.

ANTES DE CORRIGIR O SIMULADO: marque quais assuntos exigiram consulta. Concentre a próxima rodada nesses pontos. Repetir apenas questões já dominadas pode produzir sensação de preparo sem corrigir lacunas.

Gabarito do simulado - Q61 a Q66

Q61. A estratégia longa arrisca descobrir necessidades erradas tarde e gastar tempo implementando algo que perdeu relevância. Ciclos curtos permitem inspecionar uso, adaptar requisitos e limitar o investimento antes de aprender. Planejamento antecipado ajuda a explicitar restrições, dependências e estimativas quando há estabilidade. **Pontuação:** 1 ponto por riscos e comparação; 1 por feedback e vantagem contextual, sem afirmar que todo planejamento é inútil.

Q62. PO avalia valor e prioridade da demanda; Developers discutem impacto e adaptam o plano; SM apoia autogerenciamento e processo, sem agir como chefe distribuidor. Pode-se negociar escopo sem comprometer o objetivo e a qualidade, ou ordenar itens para depois. O Sprint mantém seu timebox; não se prorroga apenas para acomodar a solicitação. **Pontuação:** 1 pelos papéis; 1 por escopo, objetivo e duração.

Q63. Lead Time = $20/4 = 5$ dias. Iniciar mais trabalho não aumenta necessariamente a vazão; pode aumentar WIP e espera diante da validação limitada. Priorize concluir validações e resolver a causa da fila com colaboração e critérios de qualidade. **Pontuação:** 1 pelo cálculo com unidade; 1 pelo diagnóstico e ação.

Q64. História possível: “Como aluno, quero consultar o resultado da revisão de nota para saber a decisão e seu efeito na minha avaliação”. Aceitação: apresentar o status/decisão da solicitação pertencente ao aluno; quando concluída, exibir a resposta e o resultado definido pelas regras. DoD: revisão de código e testes relevantes concluídos, por exemplo. **Pontuação:** 1 pela história com valor; 1 por critérios específicos e DoD distinta. Não se exige a mesma redação.

Q65. Falta a hipótese de produto e a evidência que autenticação/perfil permitirá observar. Experimento: oferecer monitoria a um público delimitado, receber pedidos e organizar manualmente atendimentos pagos, medindo conversão de interessados em contratação efetiva. Defina previamente um critério de análise coerente. **Pontuação:** 1 pela distinção MVP/incremento; 1 por experimento e evidência alinhados.

Q66. $A = 200/4.000 = 5\%$; $B = 240/4.000 = 6\%$. Diferença = **1 ponto percentual**; ganho relativo = **20%**. É necessário avaliar validade da alocação e métrica, incerteza estatística e o plano do experimento; a observação de uma taxa maior não encerra a análise. **Pontuação:** 1 pelos quatro resultados; 1 pela interpretação. Esta questão não exige calcular significância.

Como corrigir respostas discursivas

As sugestões são critérios de autoavaliação, não uma rubrica oficial. Dê crédito a formulações equivalentes que preservem o raciocínio. Se você apenas nomeou o conceito sem aplicá-lo ao cenário, atribua no máximo a parcela correspondente à identificação e refaça a justificativa.

Gabarito do simulado - Q67 a Q72

Q67. Selecionar idosos representativos; usar protótipo que permita simular o fluxo necessário, até em papel; apresentar tarefa orientada ao objetivo sem ensinar telas. Justifique, respectivamente, adequação do público, possibilidade de observar interação e redução de viés. **Pontuação:** 1 pela escolha de público/artefato; 1 pela facilitação e justificativas.

Q68. A conclusão total é $4/5 = 80\%$, mas sucesso sem ajuda é $2/5 = 40\%$. O agregado esconde dependência de dicas. Investigue e corrija obstáculos, depois repita o teste. Registre caminhos errados, hesitações, tempo e entendimento da confirmação. **Pontuação:** 1 pela distinção e análise; 1 pela decisão e dois dados.

Q69. Classes distintas: **Base, Repo, Cache, Cliente e Filtro**; **CBO = 5**. m1/m2 compartilha a, enquanto m1/m3 e m2/m3 não compartilham atributos. **P = 2; Q = 1; LCOM1 = 1**. Há indício de separação de uso de estado em m3; o CBO descreve quantidade, não a qualidade das dependências. **Pontuação:** 1 por CBO e conjunto; 1 por pares, LCOM1 e interpretação.

Q70. Baixa coesão e violação de SRP: misturam-se apresentação, acesso a dados, regra e comunicação. Interface coleta/exibe; serviço coordena; domínio calcula; repositório acessa dados; componente de comunicação envia mensagens. Uma classe por linha pode produzir classitis, dispersando lógica e aumentando relações sem ganho. **Pontuação:** 1 pelo problema e divisão; 1 pelo equilíbrio entre coesão e acoplamento.

Q71. Pré-condição fortalecida: uma quantidade 5 é aceita pela base e rejeitada pela subclasse. Pós-condição enfraquecida: prazo de 7 dias satisfaz a subclasse, mas viola a promessa de no máximo 5. Um cliente que planejou receber em 5 dias perde a garantia. **Pontuação:** 1 por cada condição corretamente analisada, com contraexemplo.

Q72. Encapsular a navegação numa operação de notificação adequada reduz conhecimento interno e favorece **Demeter**. Usar contrato de envio com implementações/adaptadores de fornecedores favorece **DIP**. Abstrações sem necessidade podem aumentar classes, indireções e custo de entendimento; devem representar variações reais. **Pontuação:** 1 pelas duas melhorias/princípios; 1 pela justificativa e limitação.

Sua próxima revisão

20 a 24 pontos: refaça erros e explique oralmente as diferenças centrais. **14 a 19:** revise os capítulos relacionados às perdas e faça novas tentativas sem consulta. **Até 13:** retome exemplos resolvidos e pratique os cálculos e cenários em blocos menores. Essas faixas orientam estudo; não estimam sua nota na prova real.

Antes de encerrar, consiga explicar: valor antes de implementação; feedback antes de ampliar investimento; qualidade antes de declarar pronto; dependências explícitas; evidência antes de conclusão. Esses raciocínios conectam grande parte do conteúdo da disciplina.